G1, ZGC, Shenandoah… Avec tous ces GCs dans Java, je choisis lequel ?

Antoine DESSAIGNE

Les GCs dans Java en 2026…

G1	ZGC	Shenandoah	Serial	Parallel

Comprendre les GCs
(Garbage Collectors)
(en Java)

Antoine DESSAIGNE

axway

74Software

À quoi sert un GC ?

Allouer
la mémoire

Libérer
la mémoire

À quoi sert un GC ?

Allouer
des objets

Libérer
des objets

La mémoire ? Les objets ?

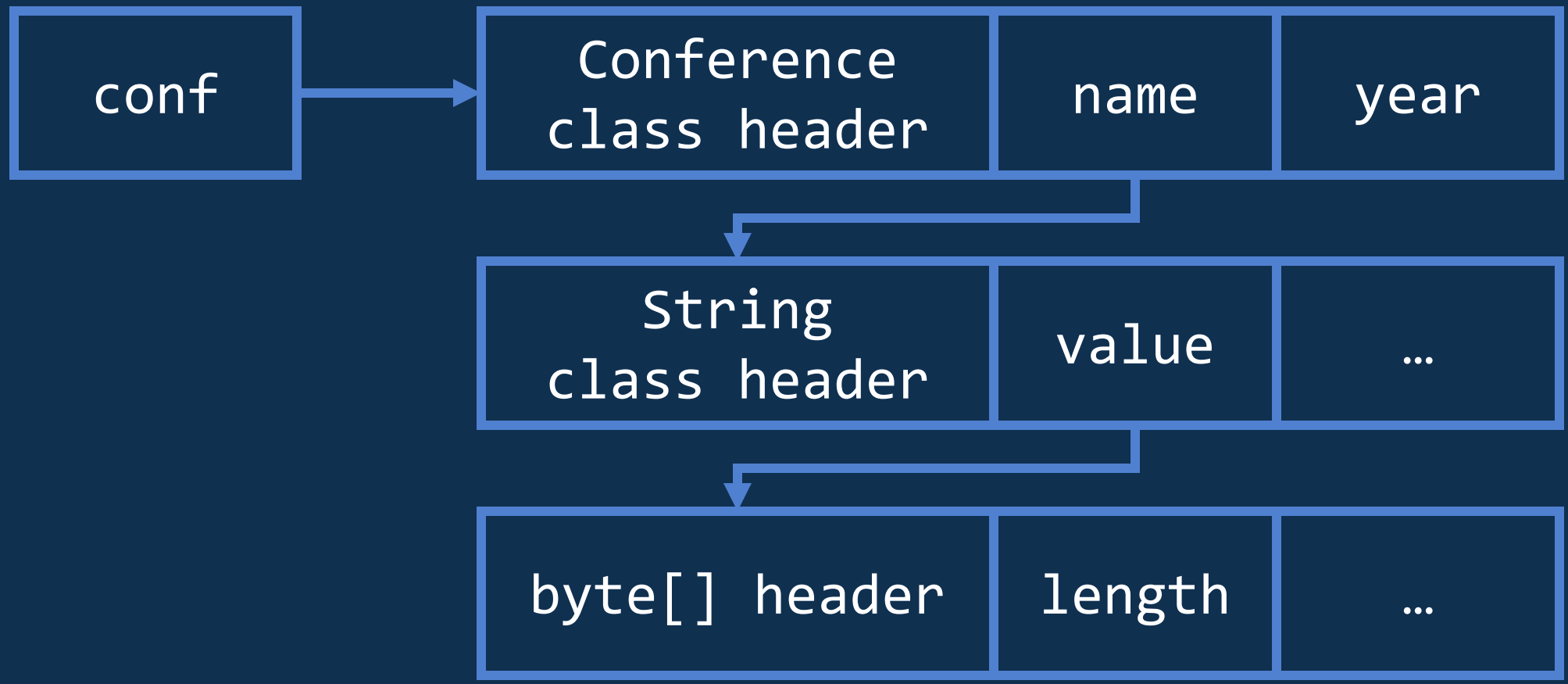
```
var conf = new Conference("Conférence", 2026);
```

La mémoire ? Les objets ?

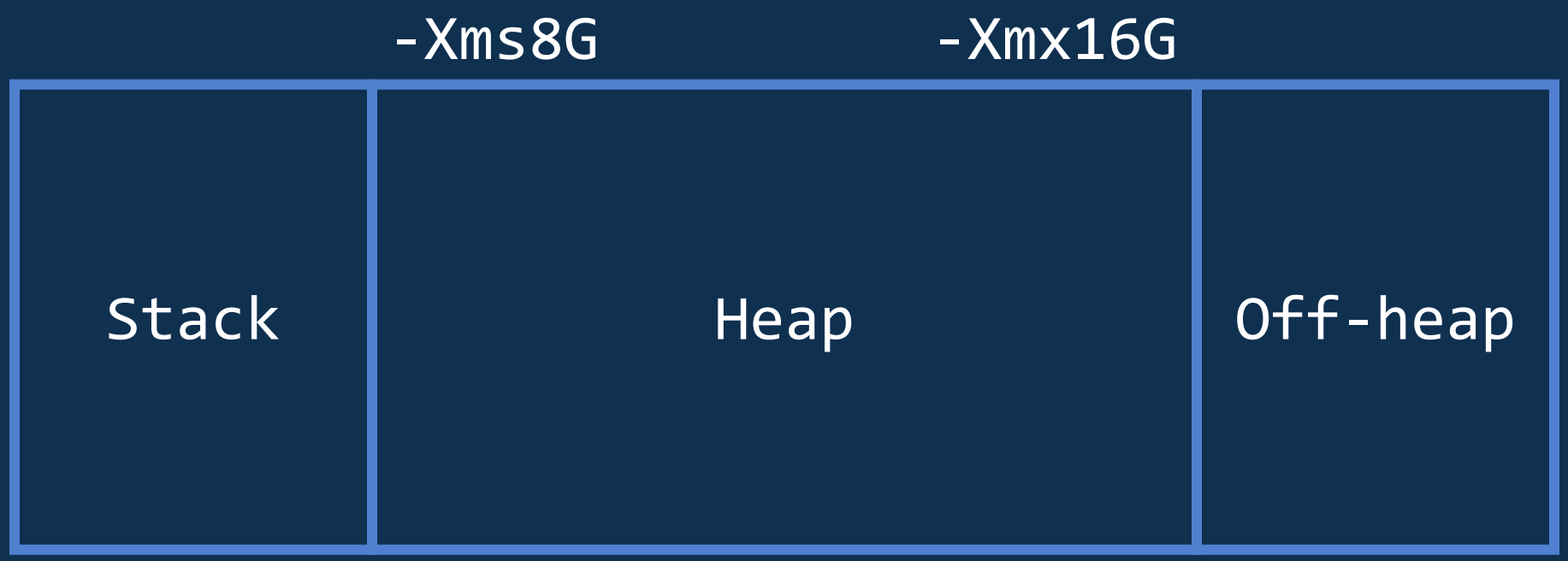
```
var conf = new Conference("Conférence", 2026);
```

La mémoire ? Les objets ?

```
var conf = new Conference("Conférence", 2026);
```



La mémoire dans Java



Au commencement...

Java 1

Processeur : 100 à 166 Mhz

Mémoire : 8 à 16 Mo

Stockage : 500 à 1200 Mo

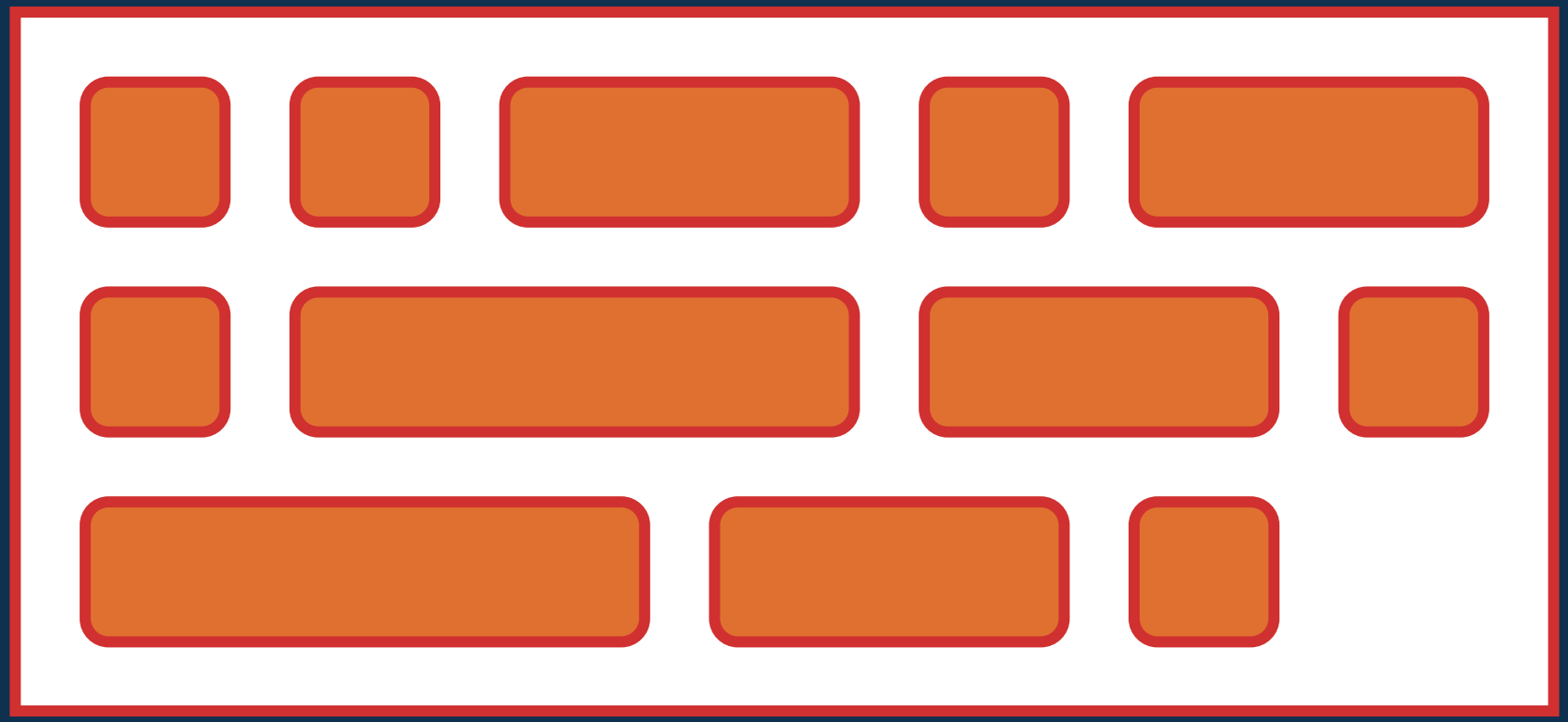
Bref…

Java 1

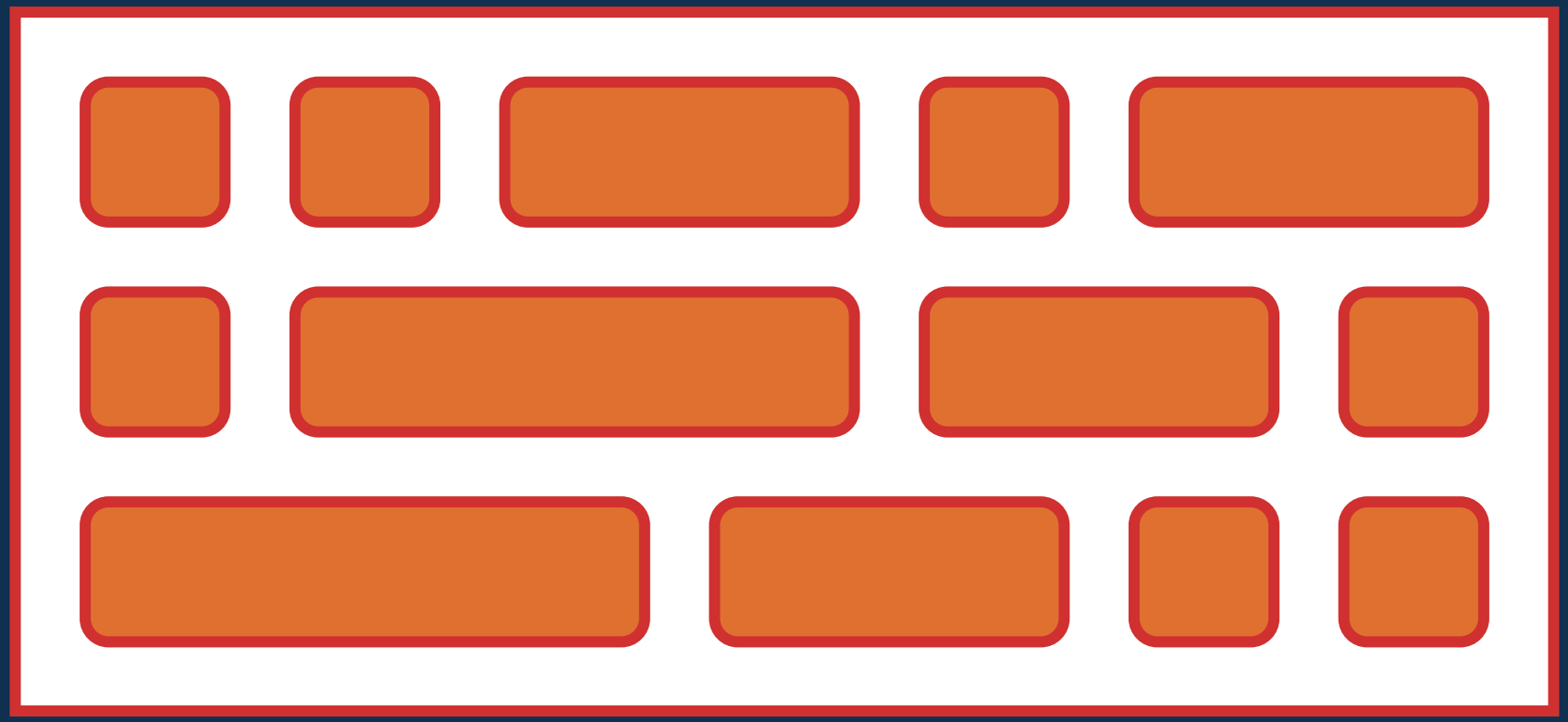
Java 1

Java 1

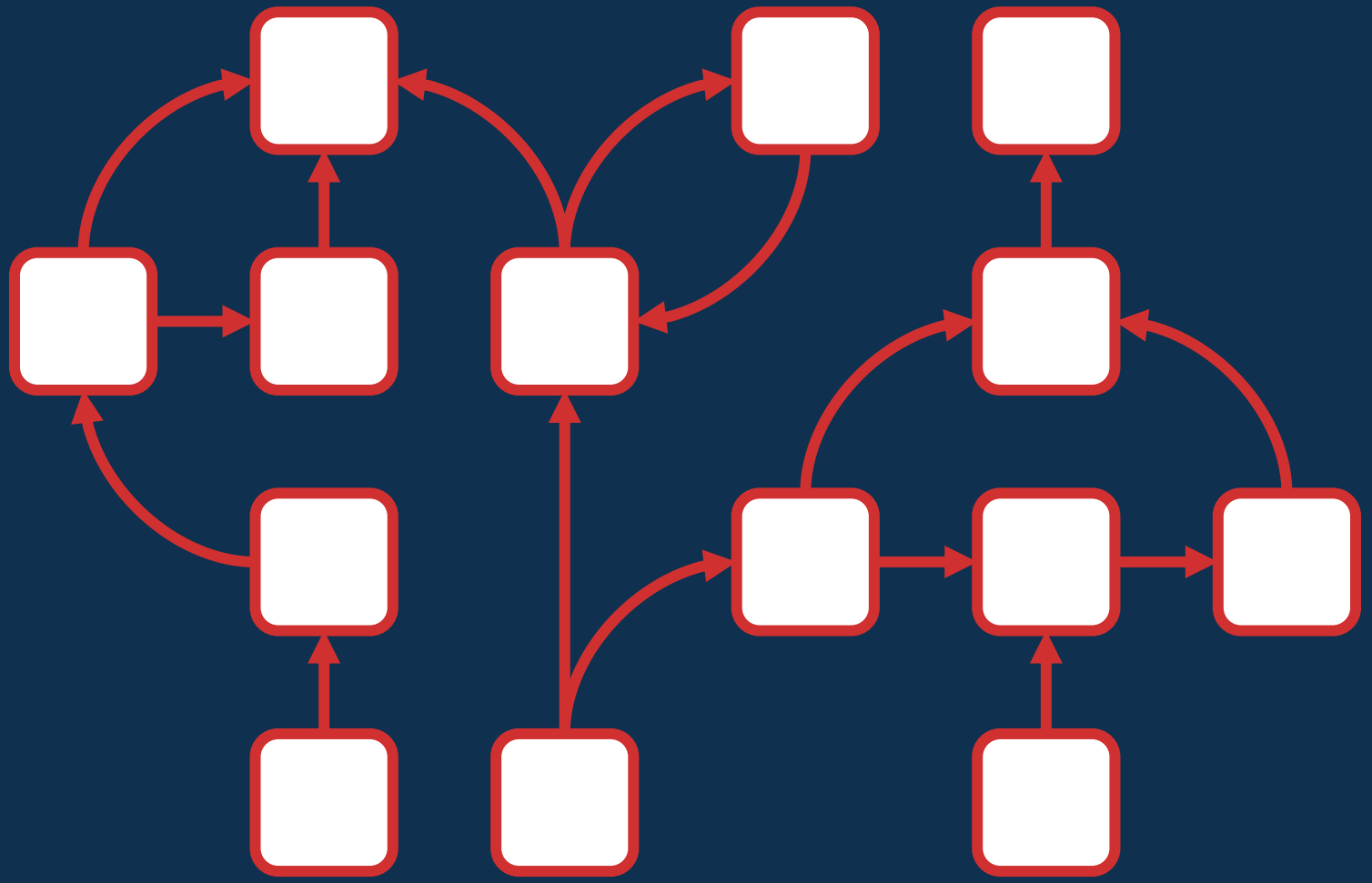
Java 1

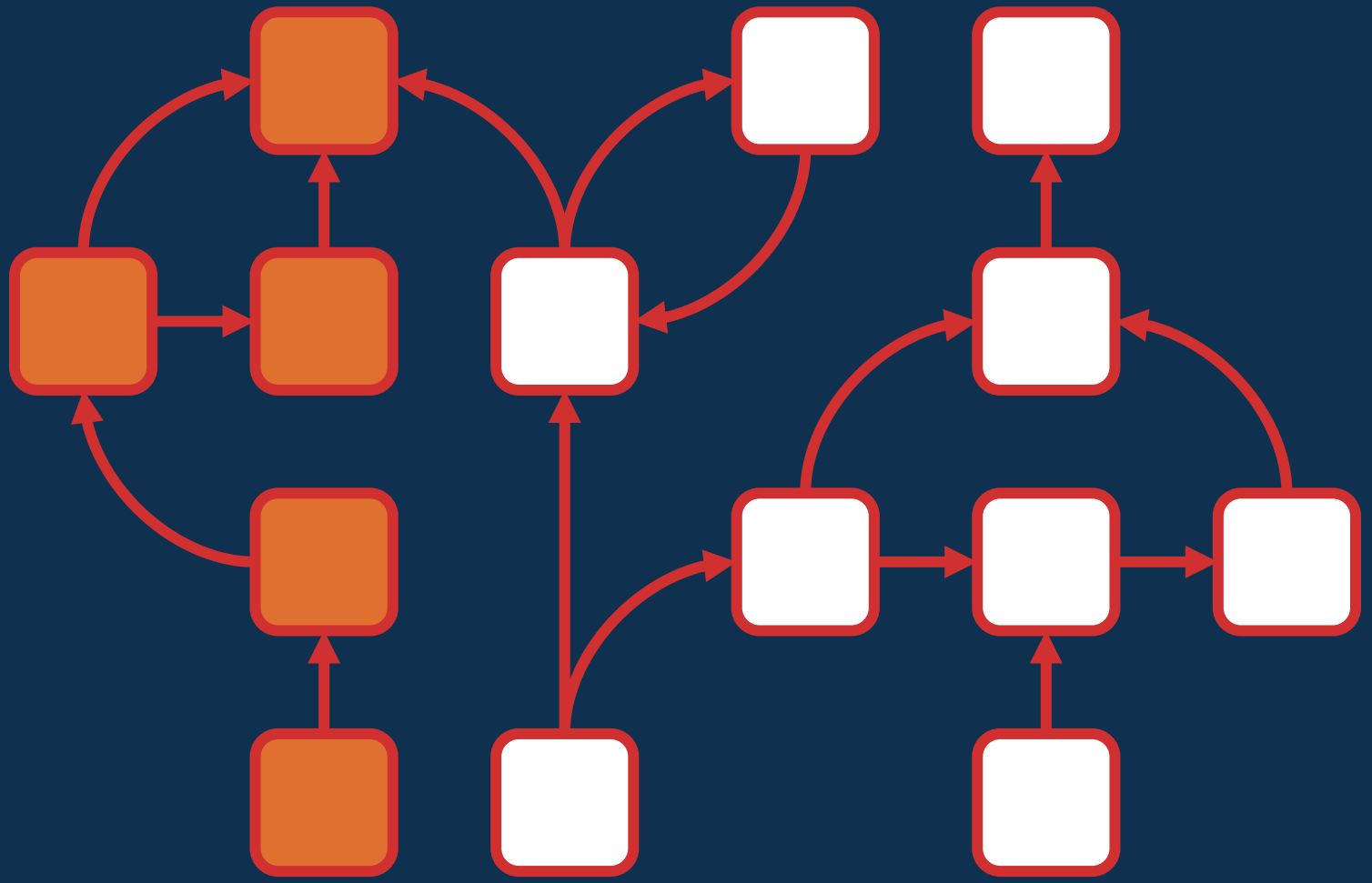


Java 1

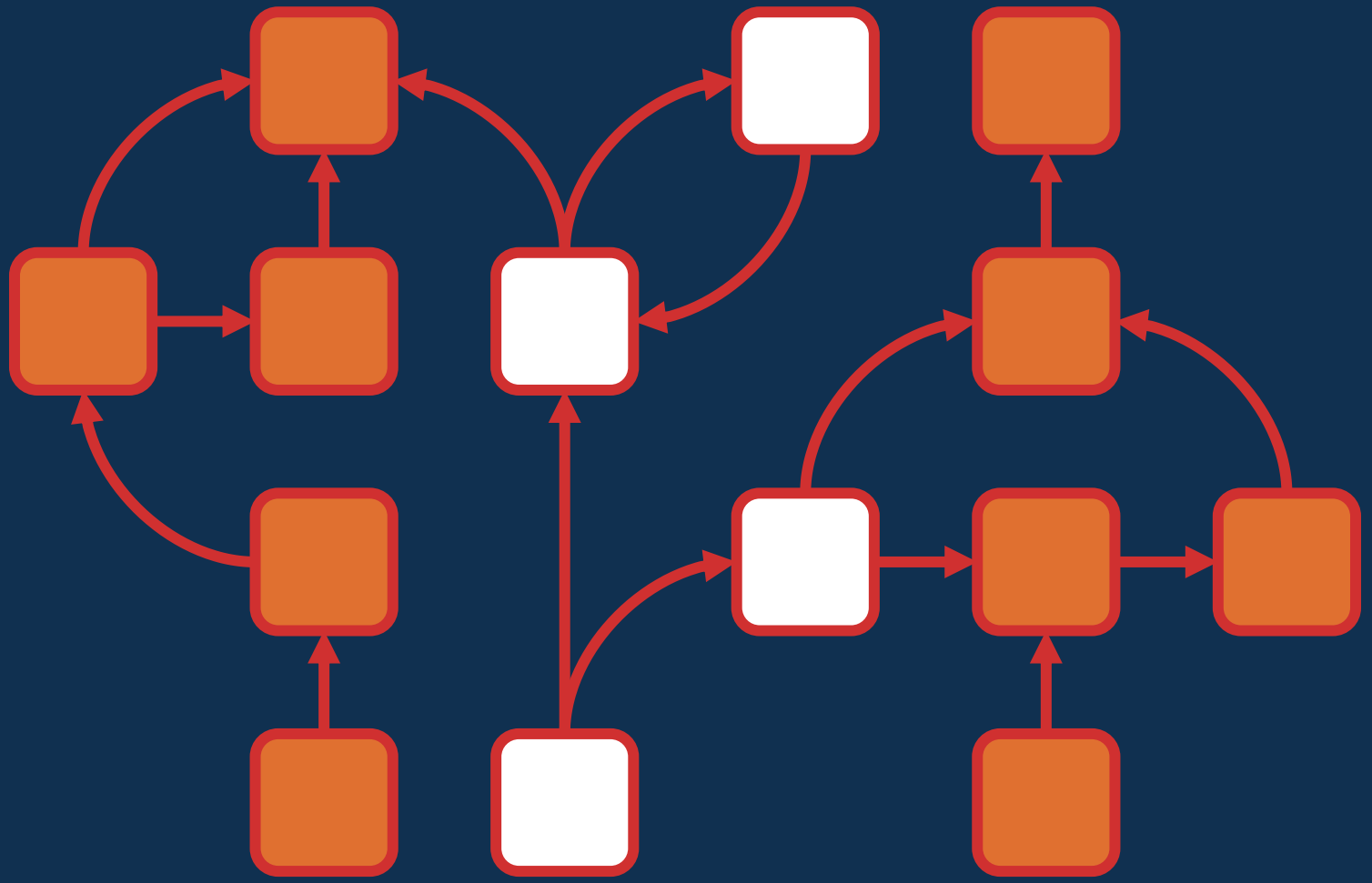


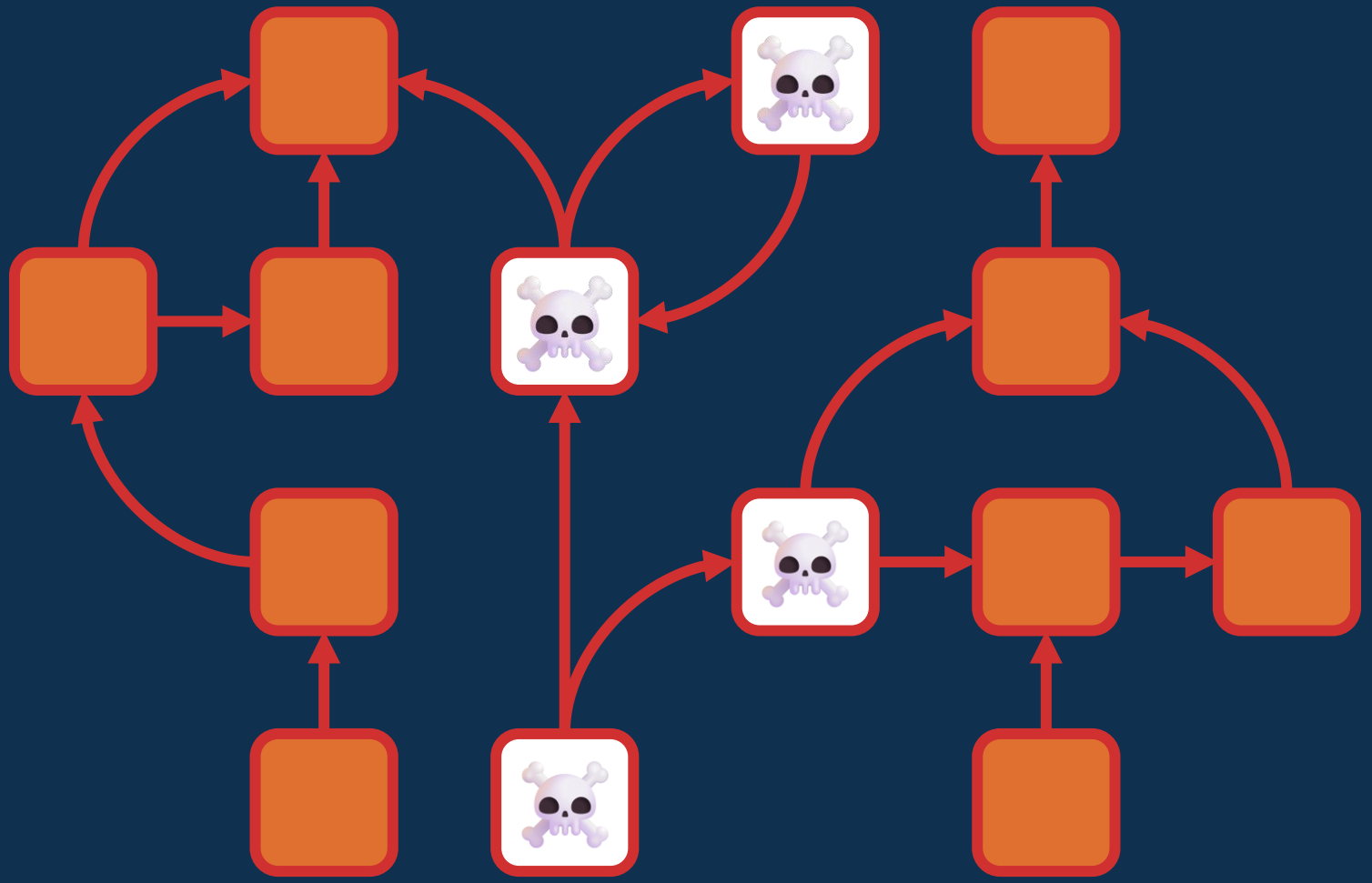
Comment on sait si un objet est toujours utilisé ?





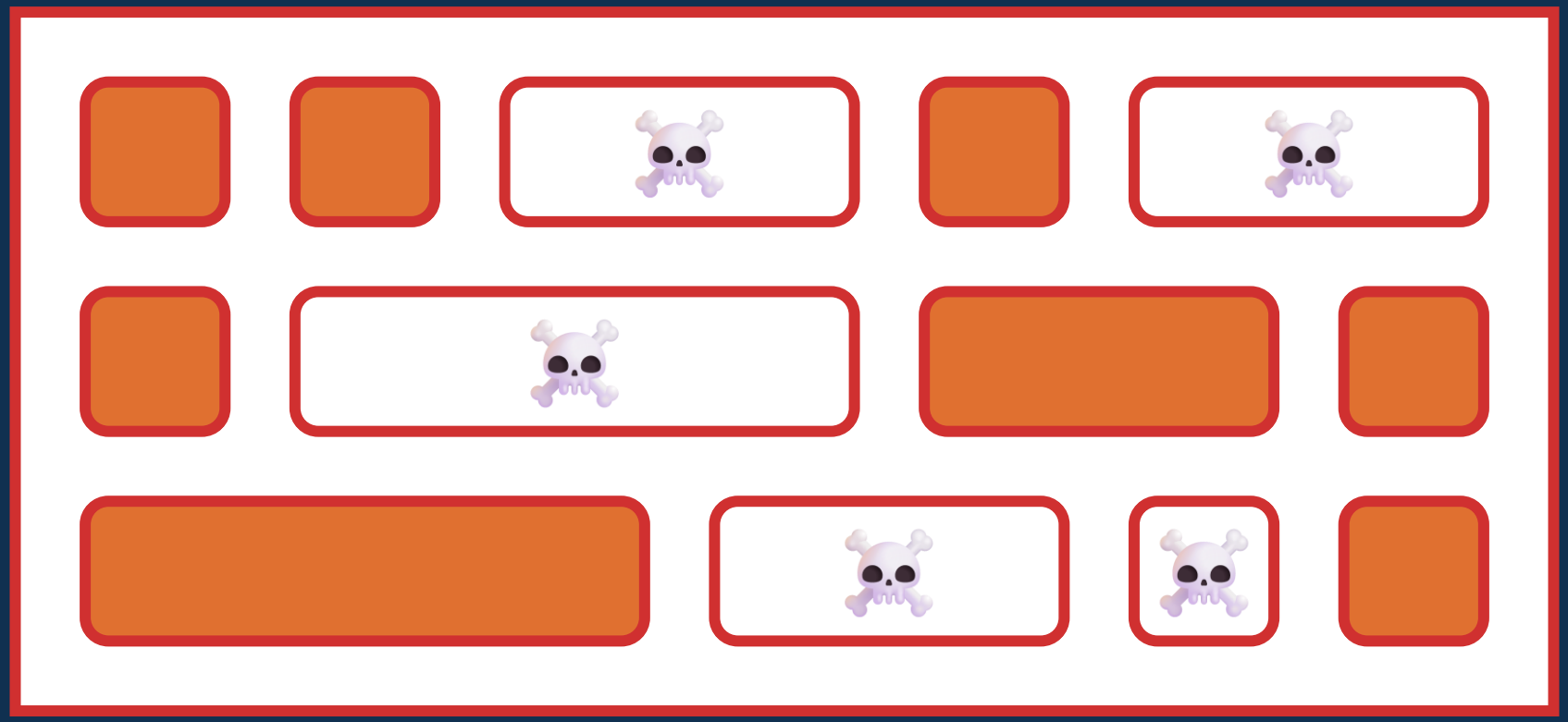
Local variables


Local variables
Static fields

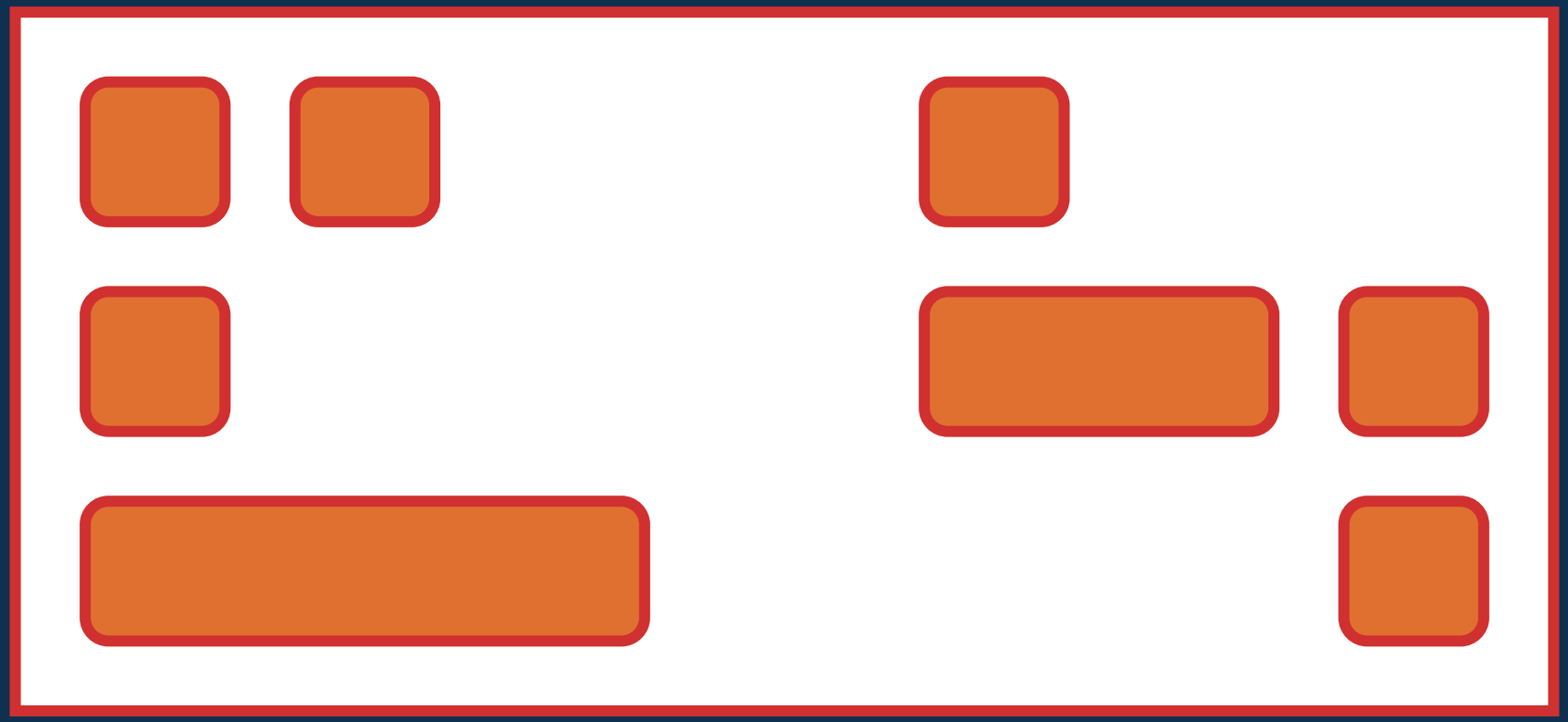


GC roots = Local variables + Static fields

Java 1



Java 1



Java 1

- 👍 Ça marche et c'est déjà bien
- 🧀 Ça fait des trous

Ce serait bien d'éviter de parcourir tous les objets à chaque fois…

L'hypothèse générationnelle

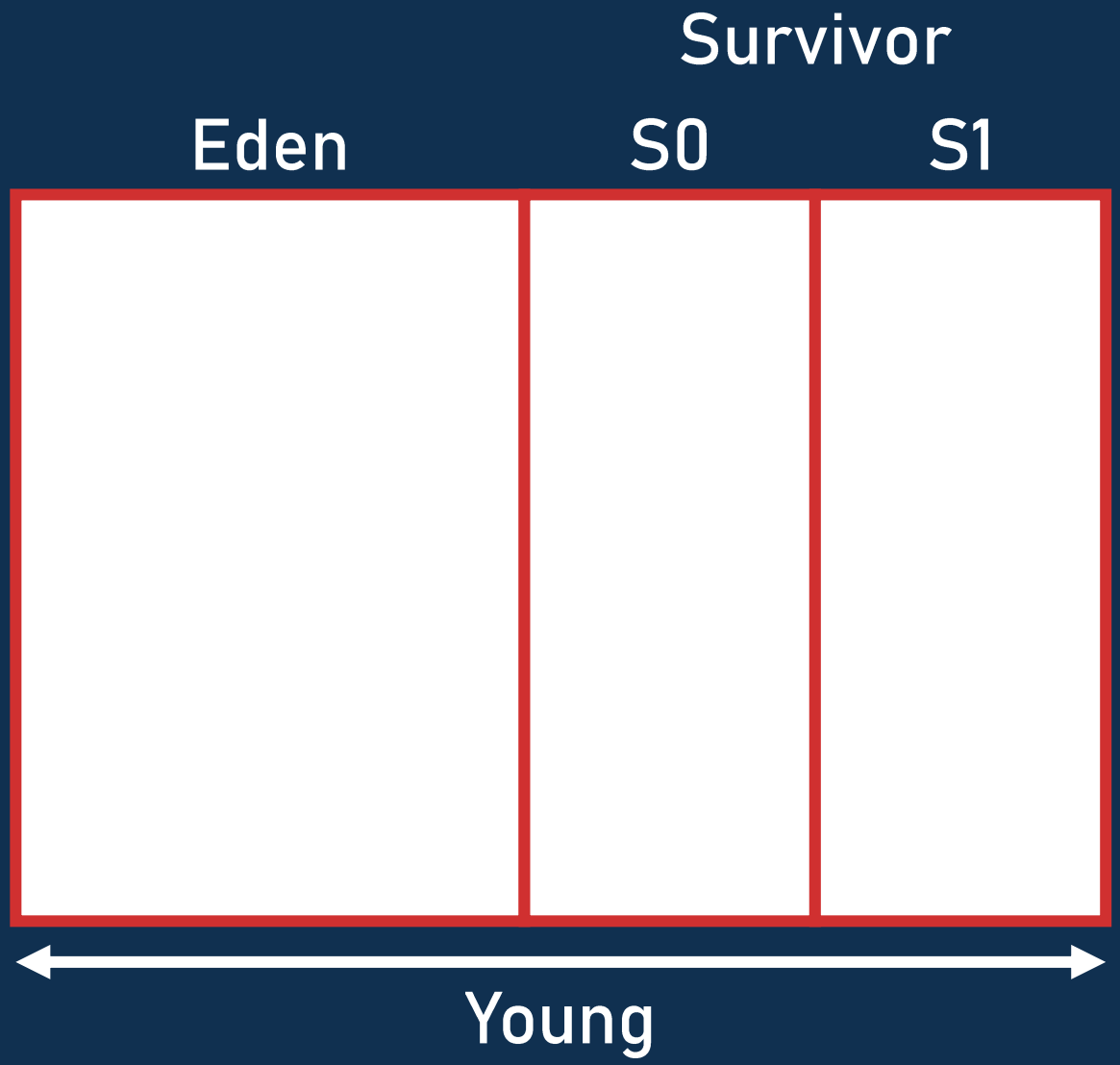
L'hypothèse générationnelle

- ☠️ Soit les objets meurent rapidement
- ♾️ Soit les objets sont là pour longtemps (ou toujours)

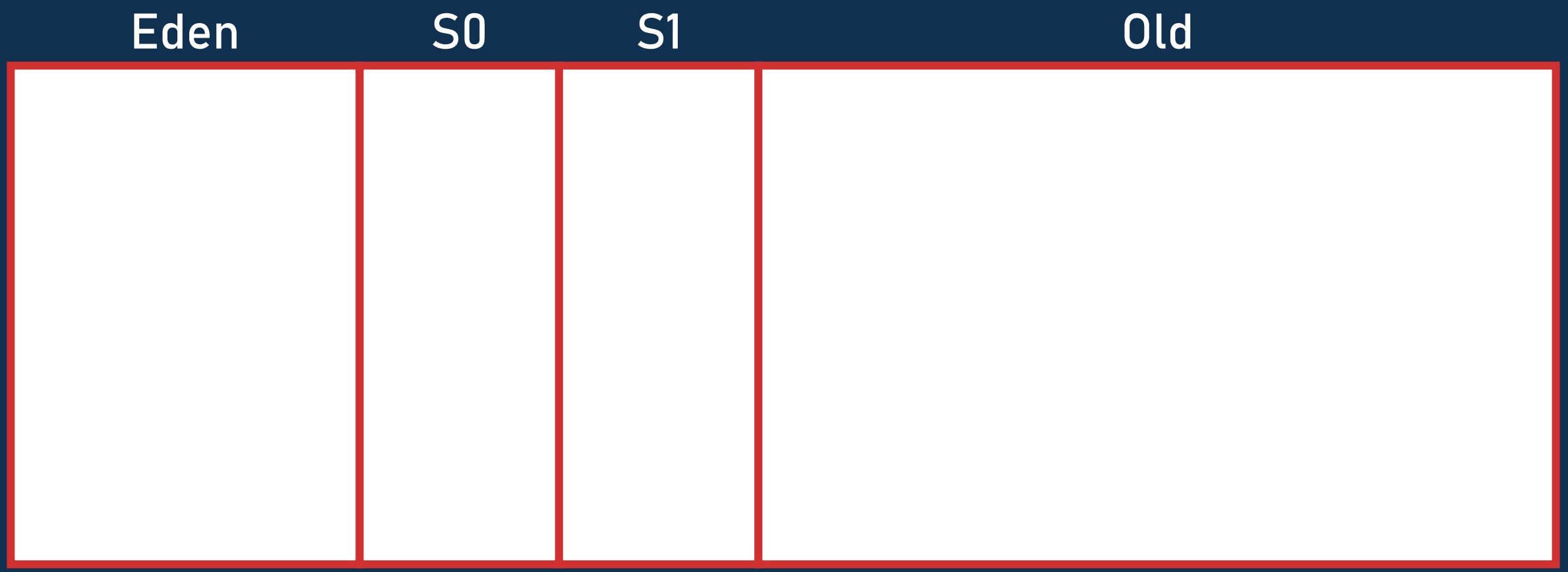
Serial GC

Java 1.2 (1998)

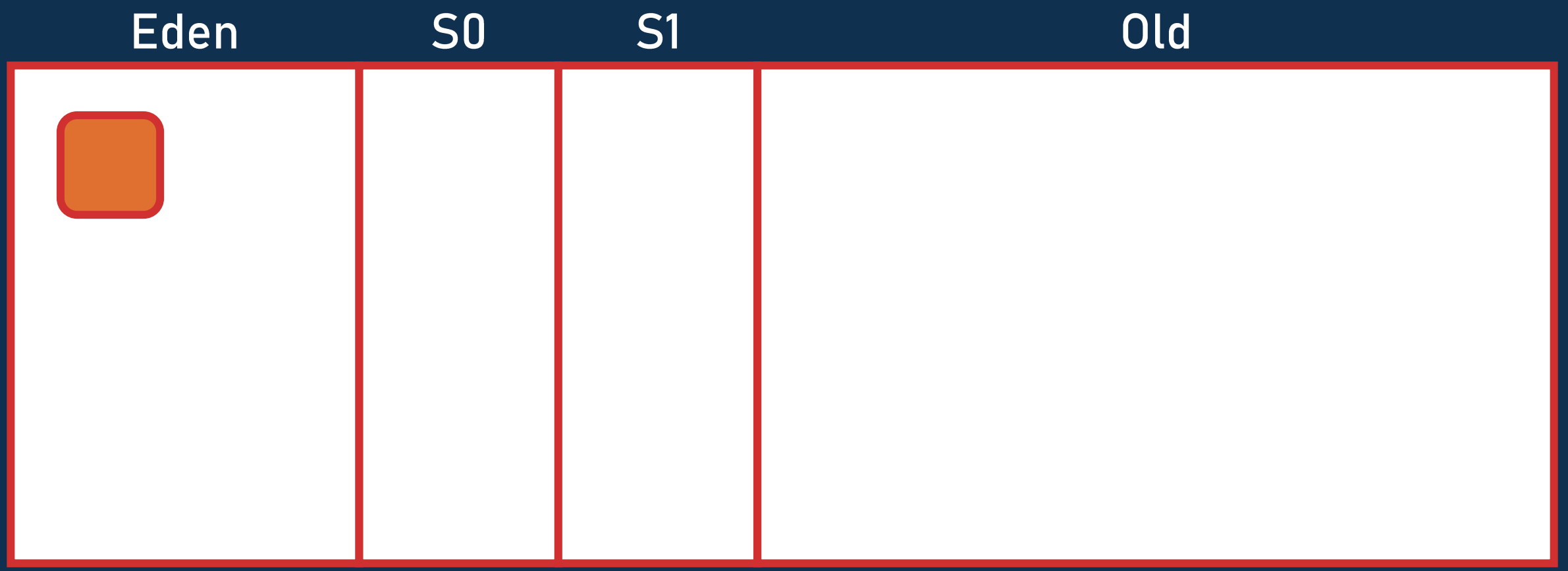
Serial GC



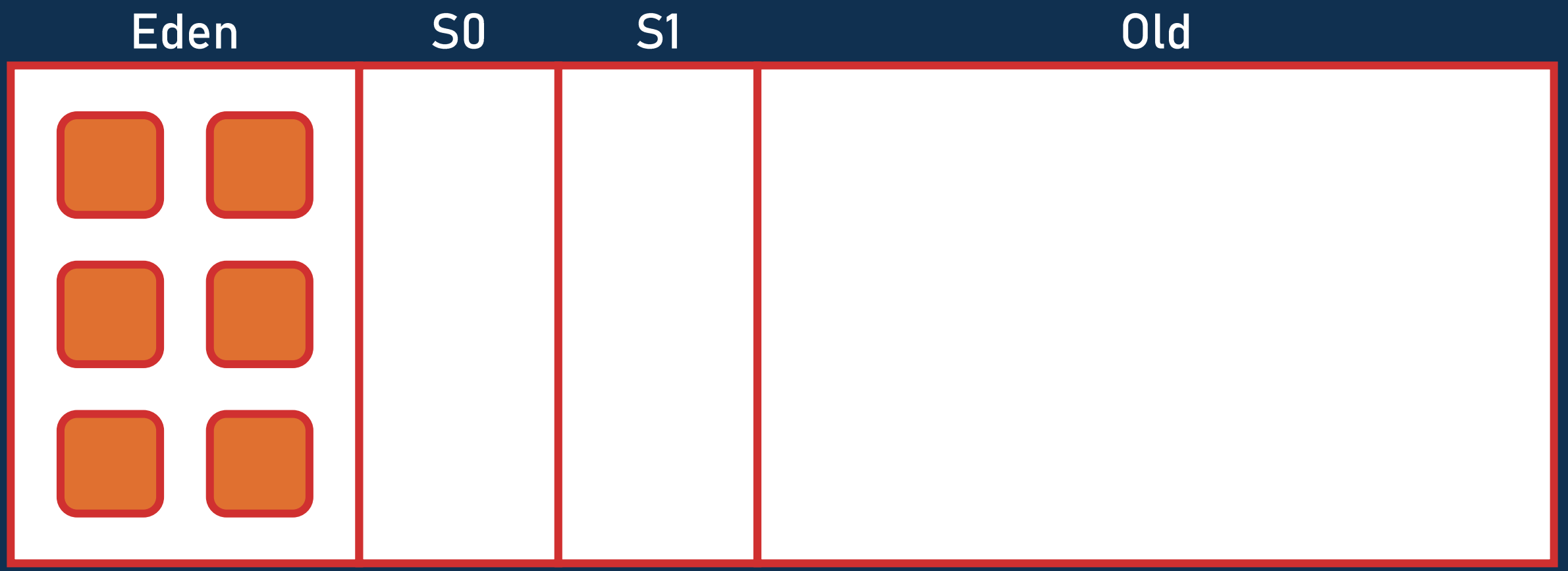
Serial GC



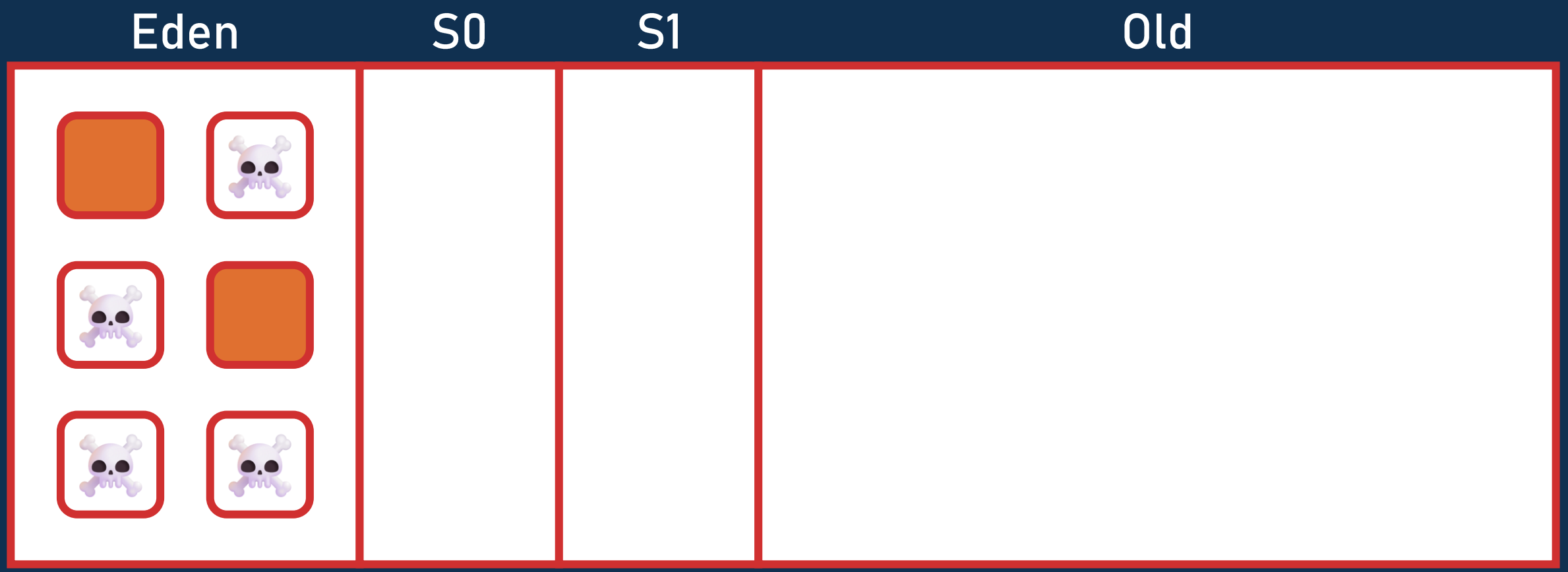
Serial GC



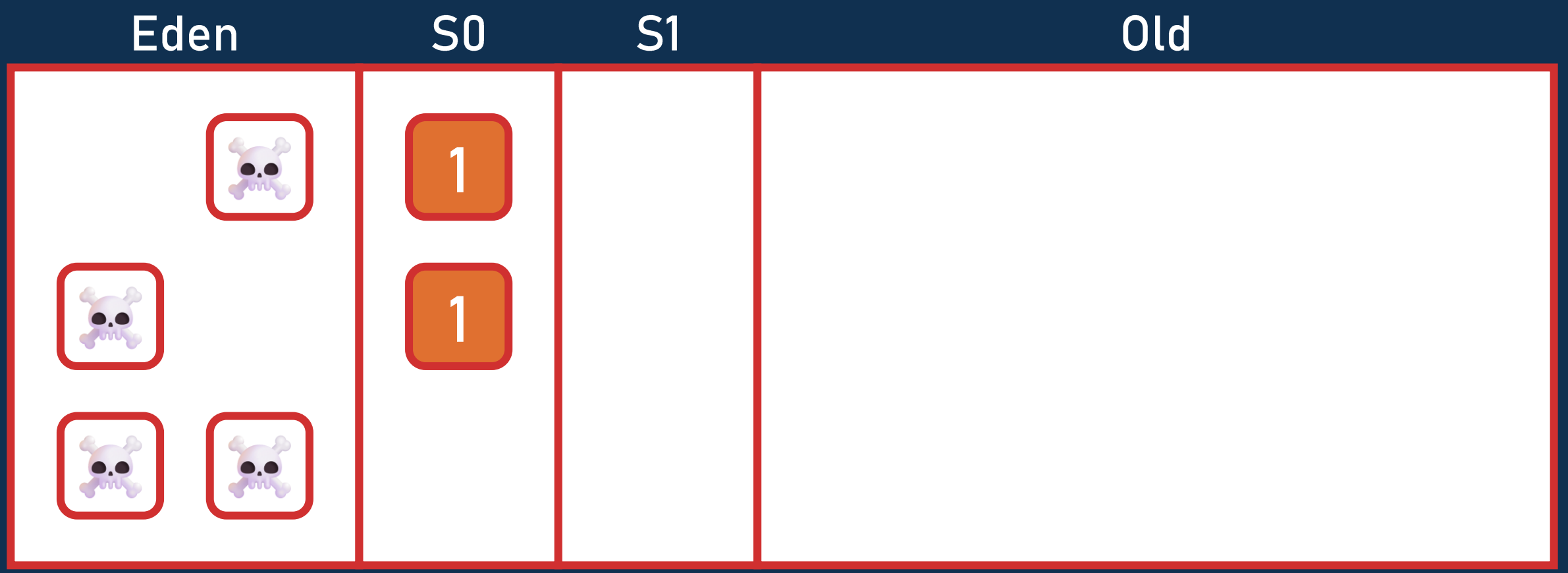
Serial GC



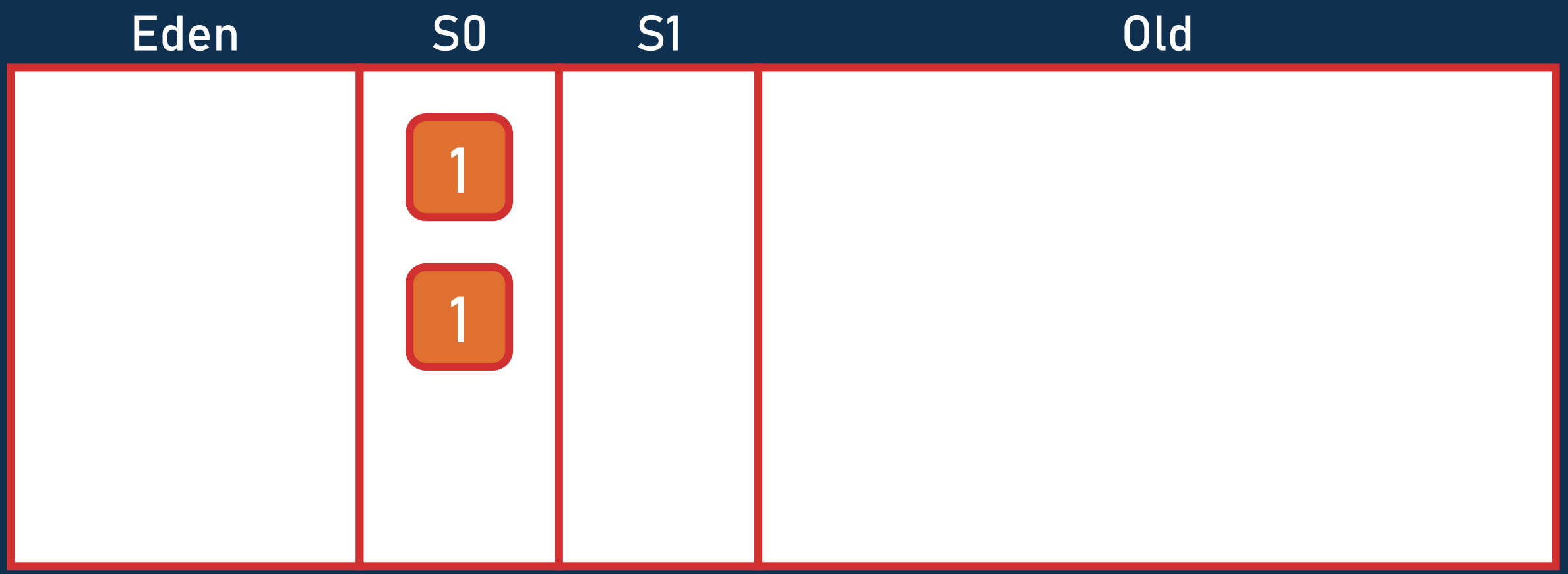
Serial GC



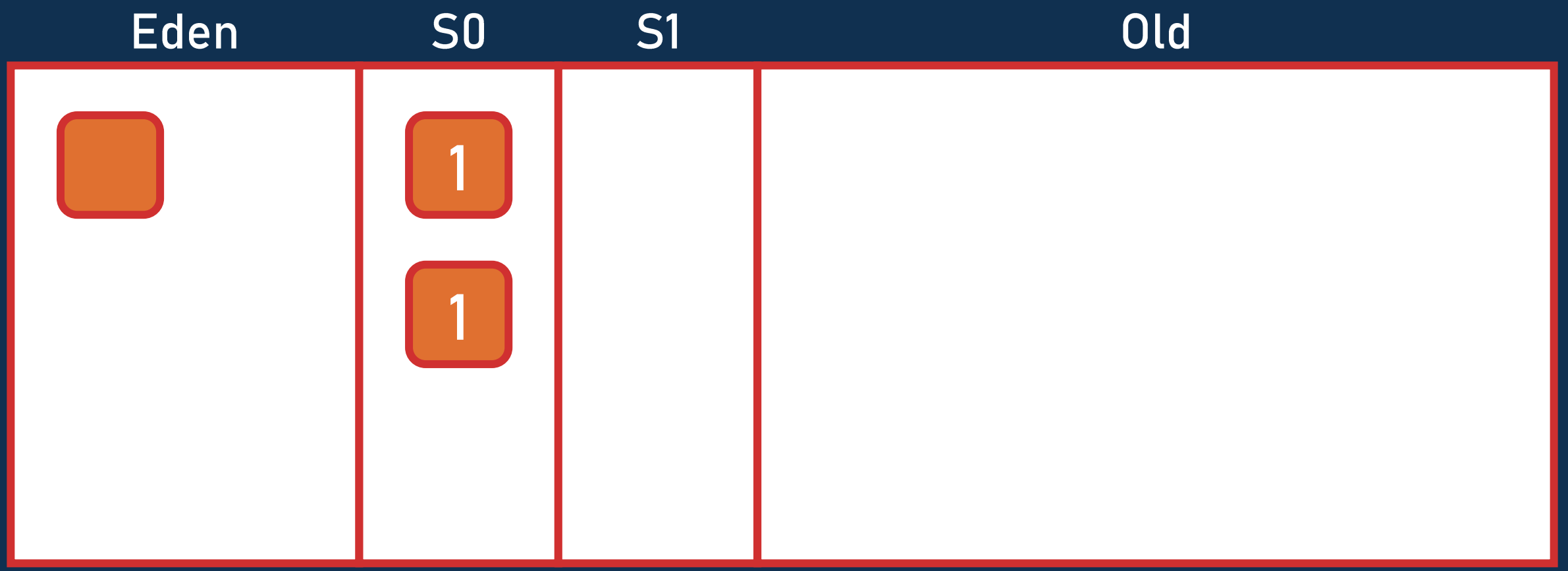
Serial GC



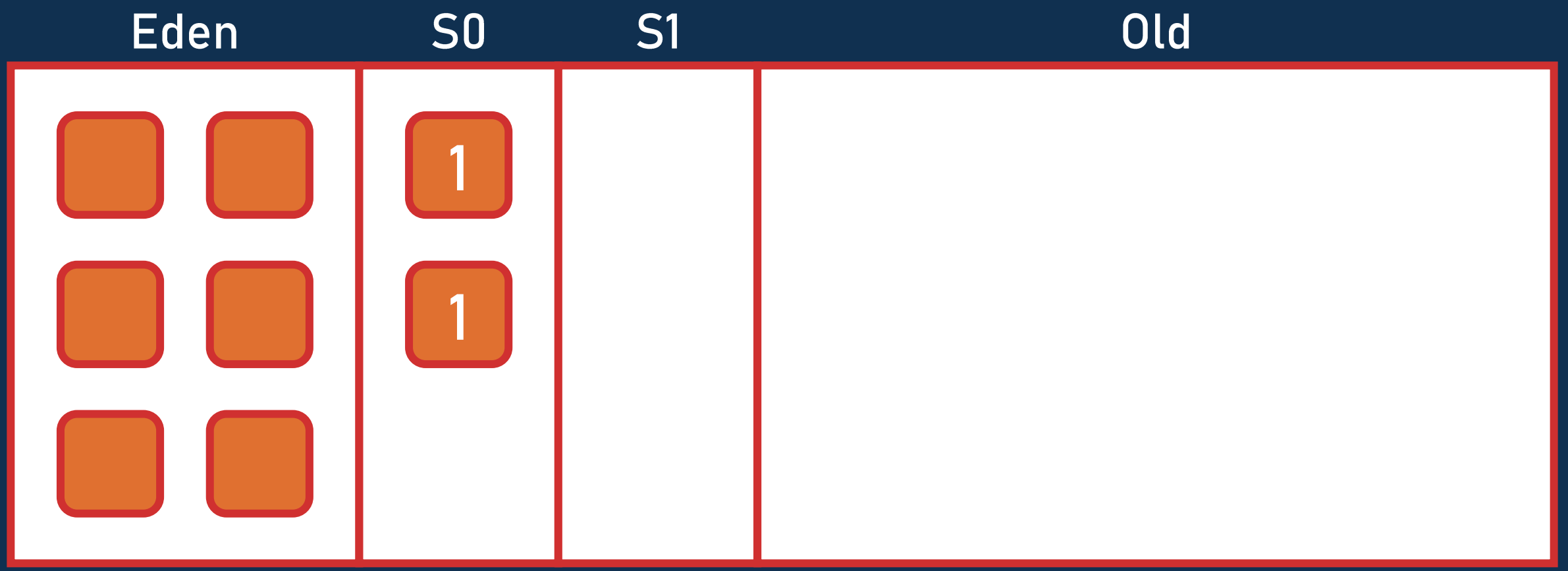
Serial GC



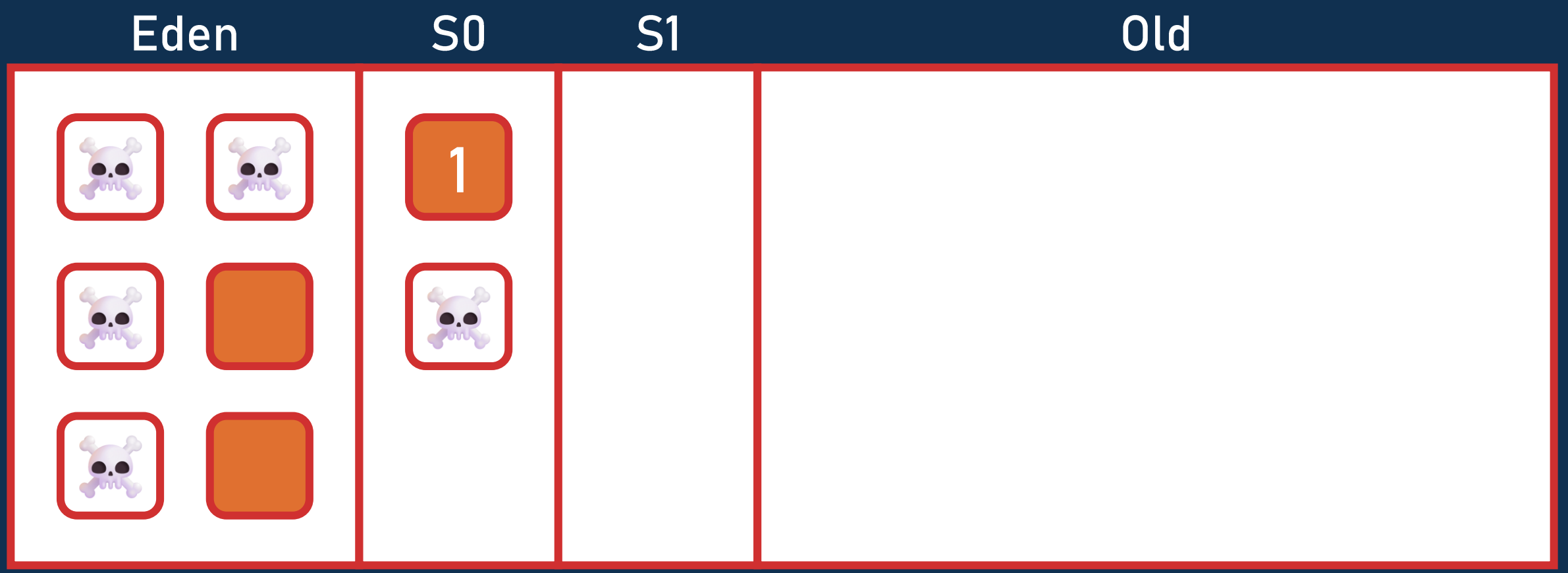
Serial GC



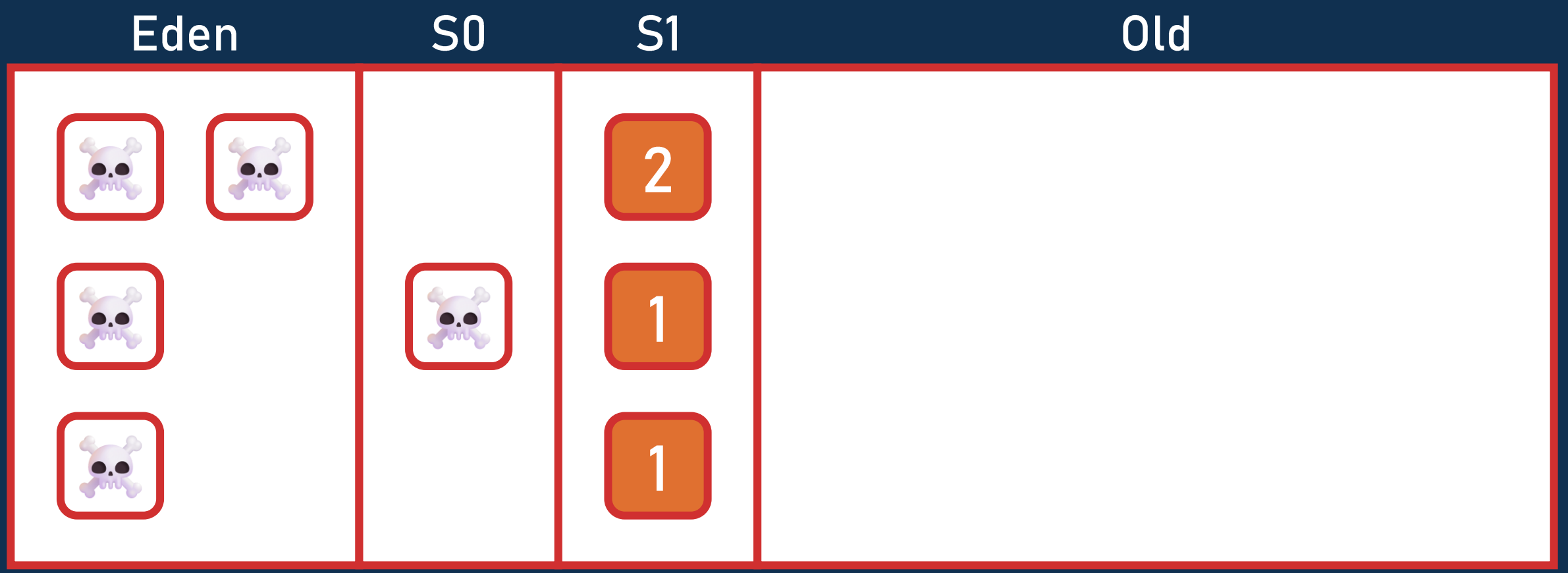
Serial GC



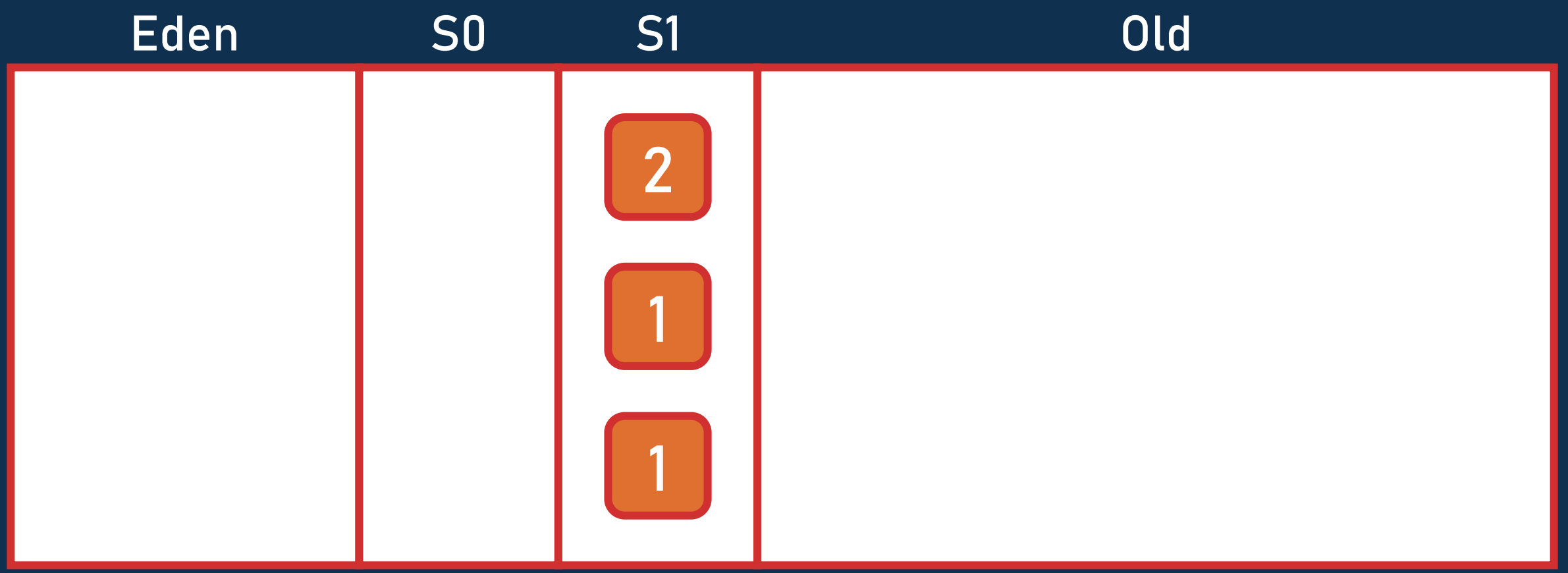
Serial GC



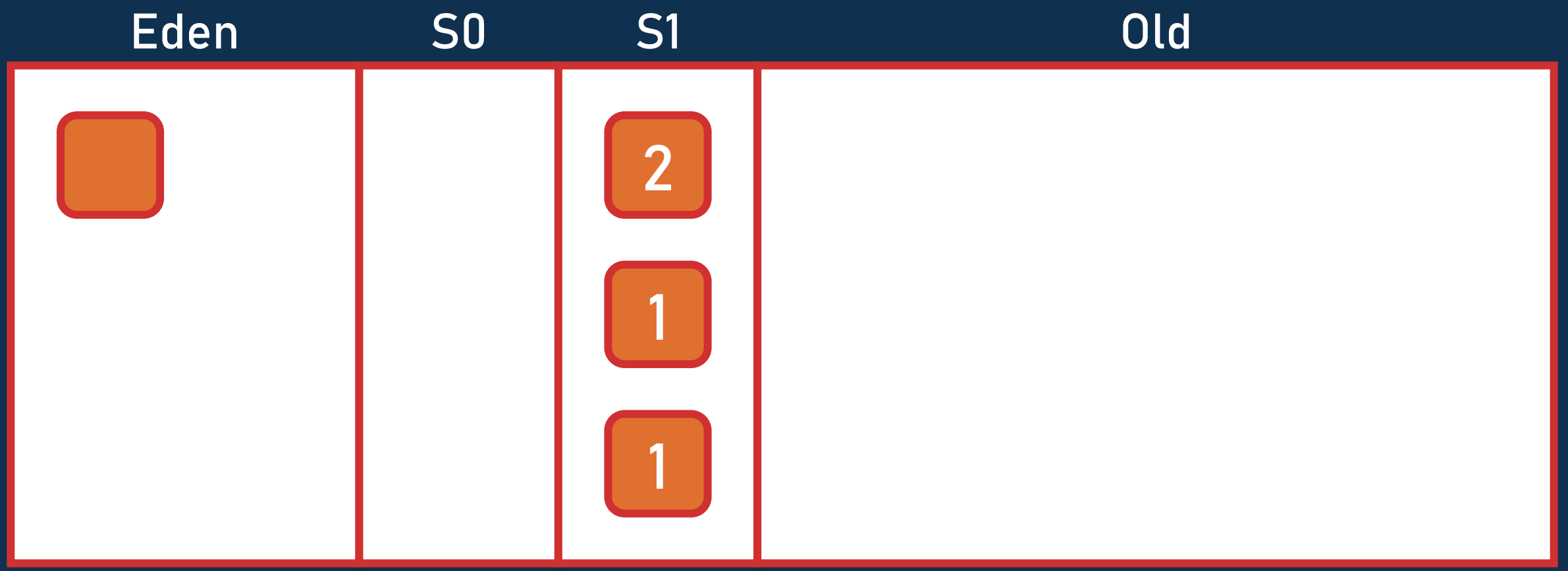
Serial GC



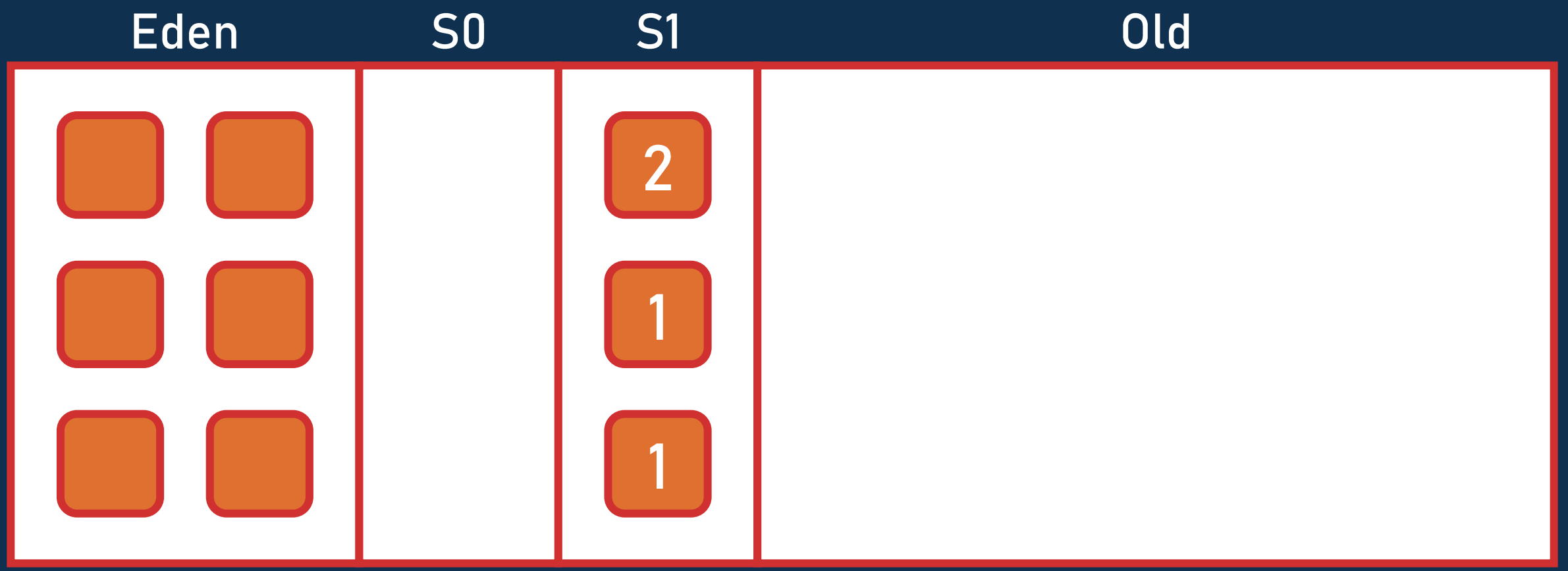
Serial GC



Serial GC

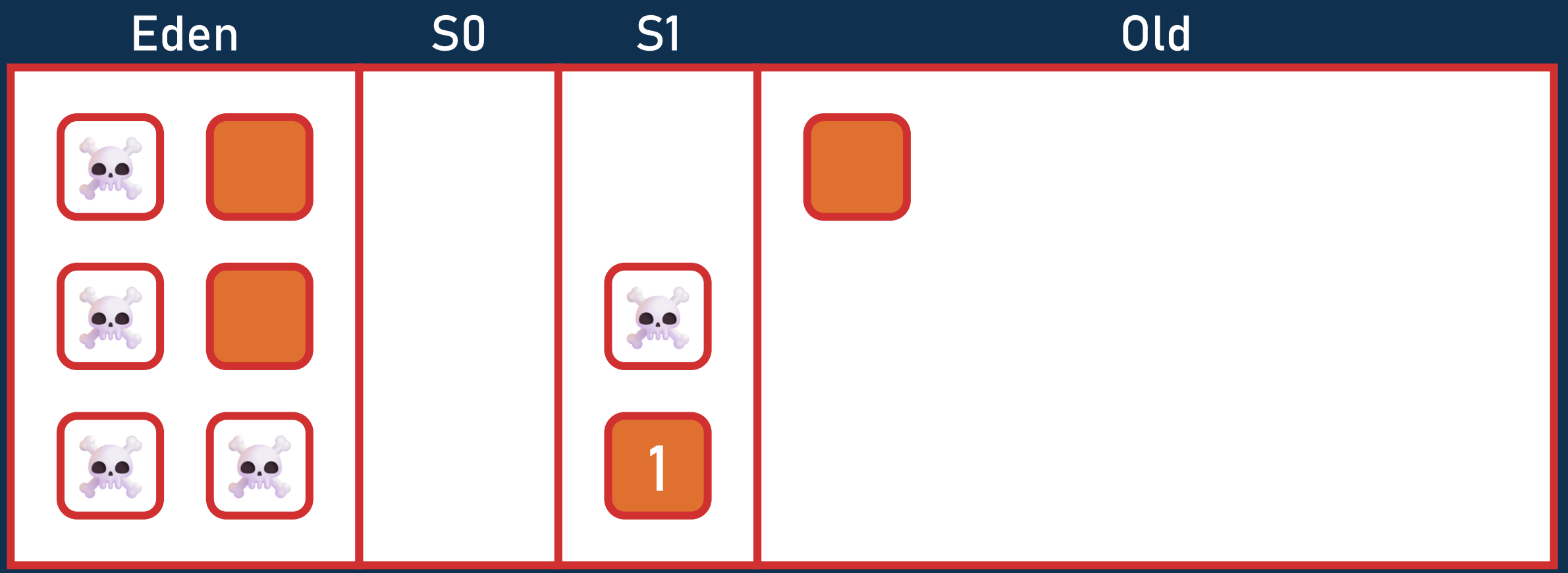


Serial GC

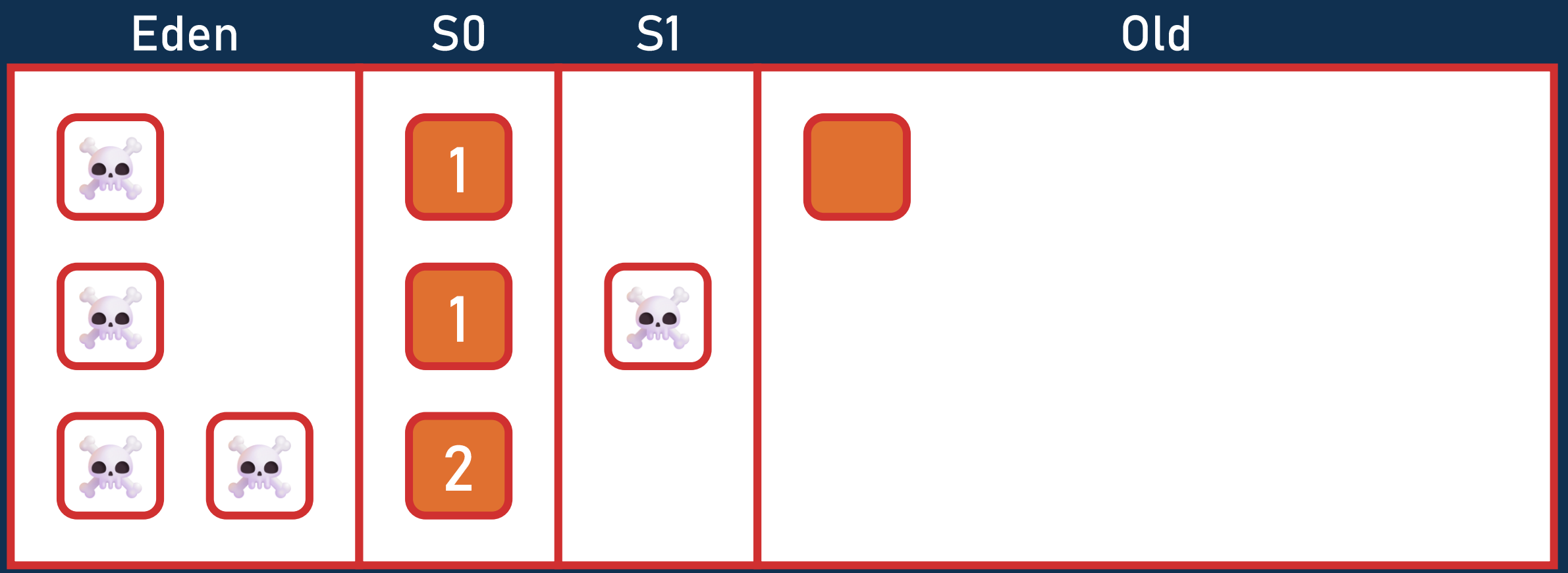


Serial GC

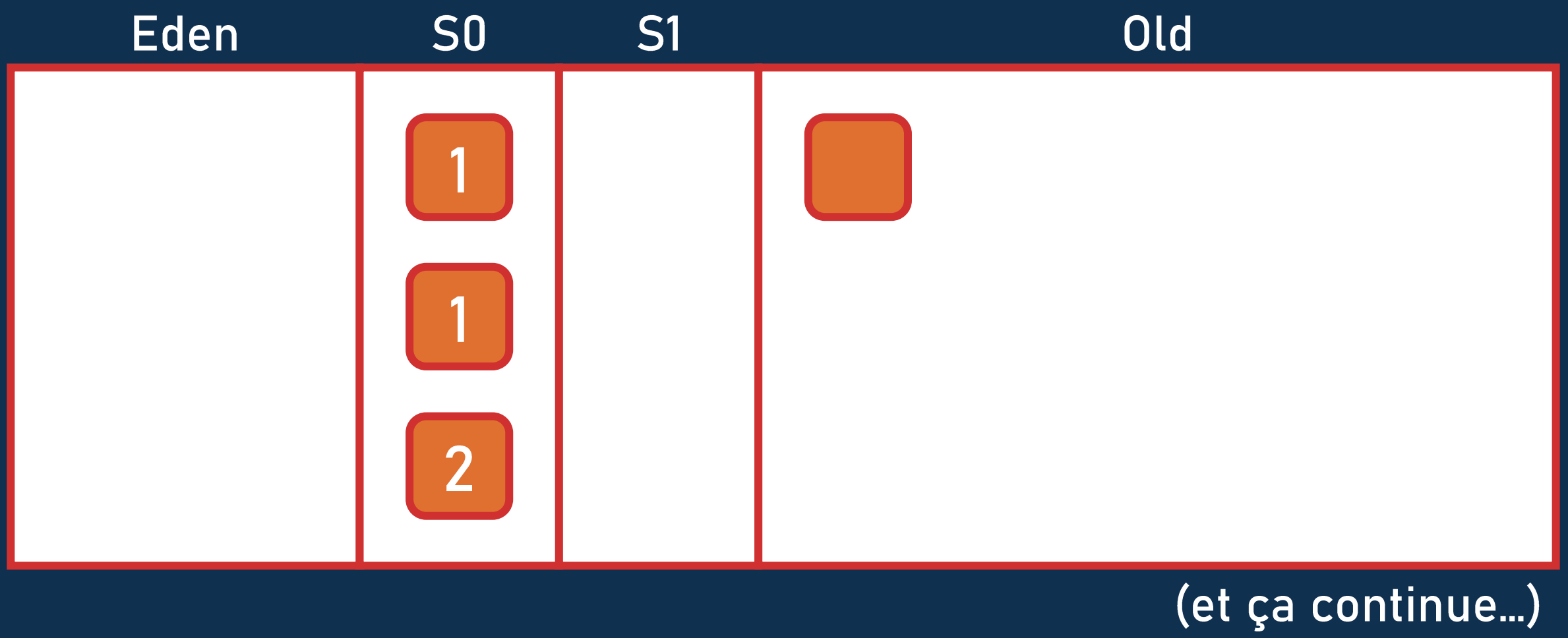
Serial GC



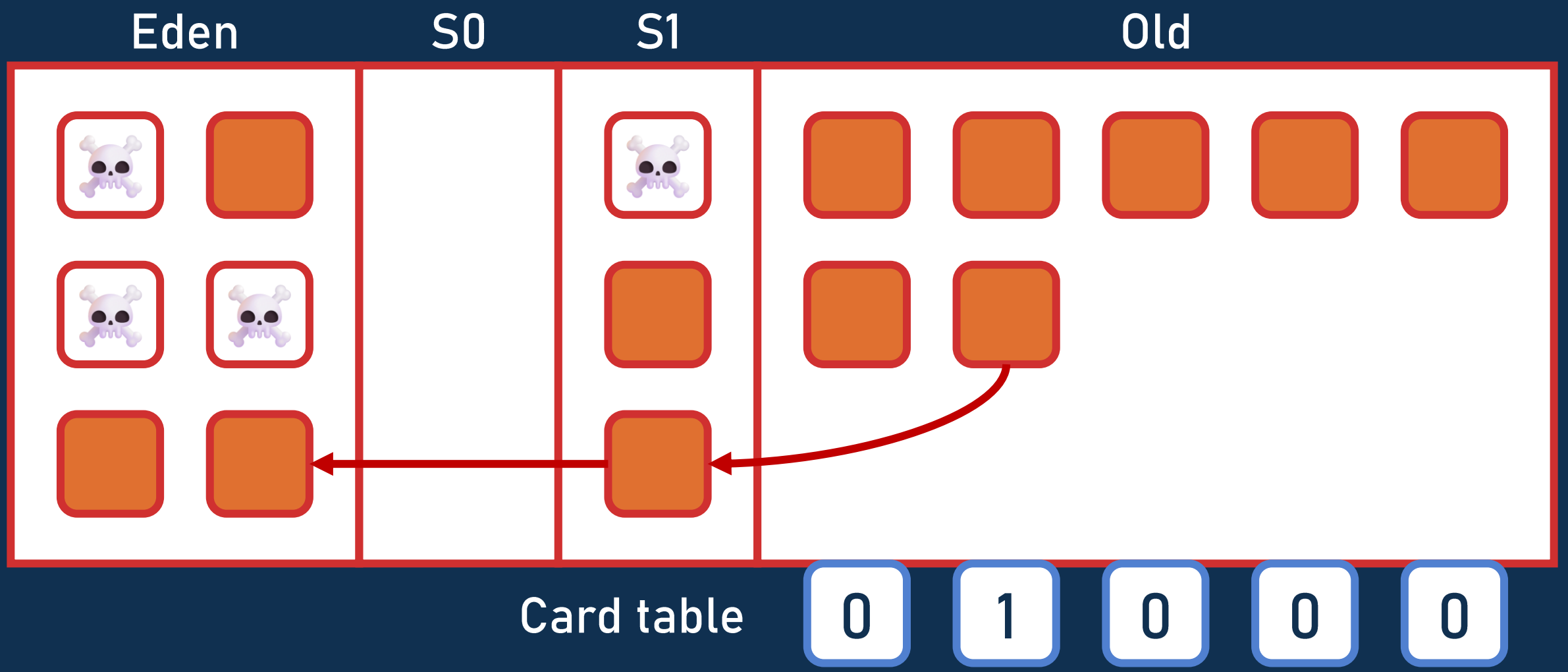
Serial GC



Serial GC



Serial GC

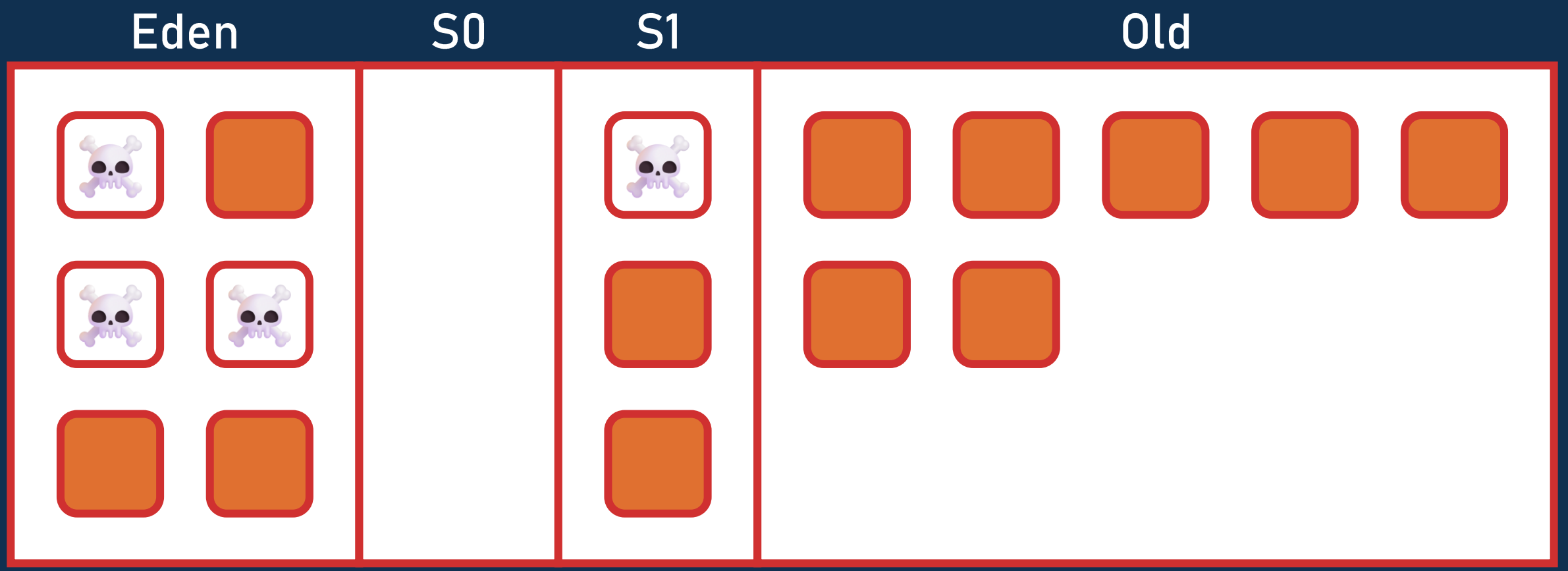


Serial GC – Write barrier

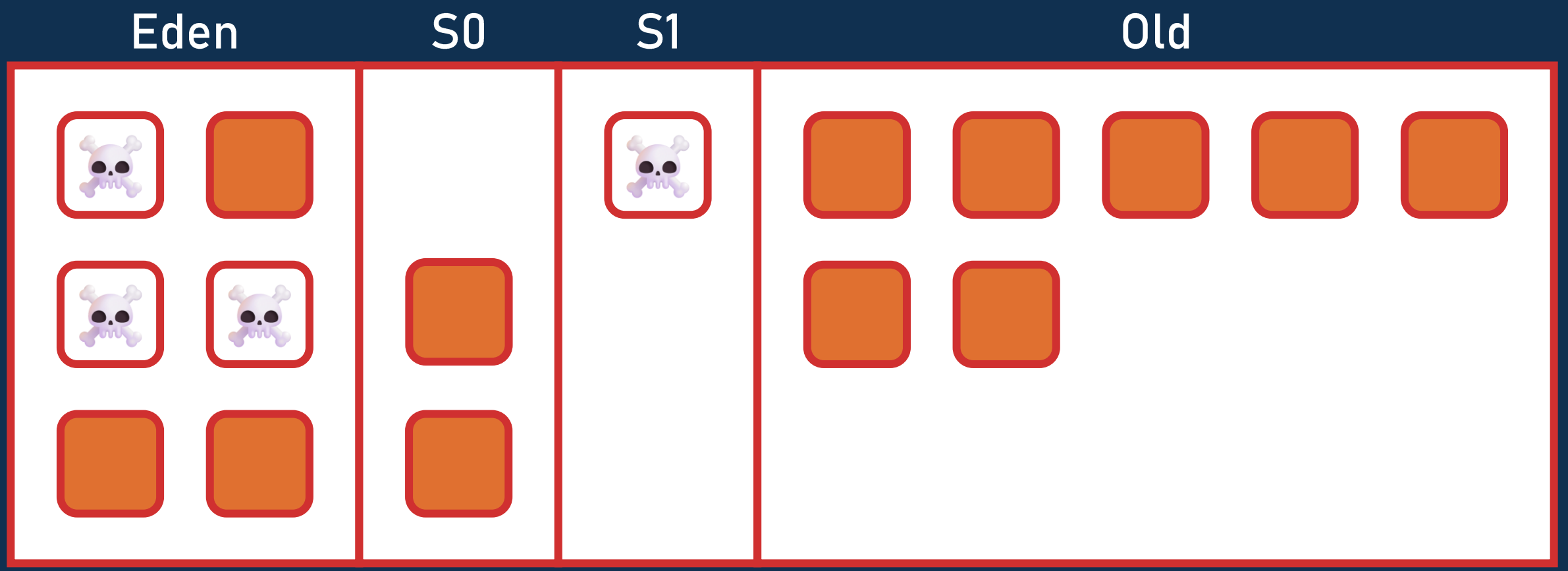
```
cache.addValue(value);


if (isOldObject(cache) && !isOldObject(value)) {
    int index = getMemoryAddress(cache) / 512;
    cardTable[index] = 1;
}
```

Serial GC



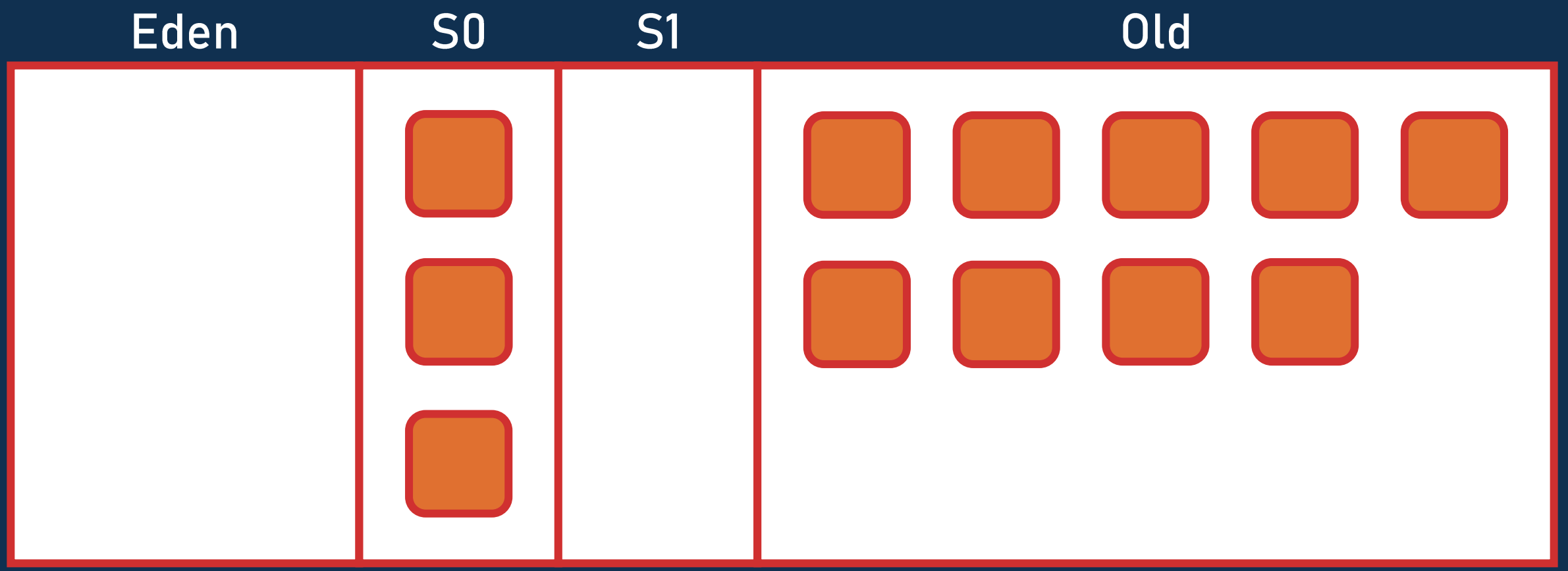
Serial GC



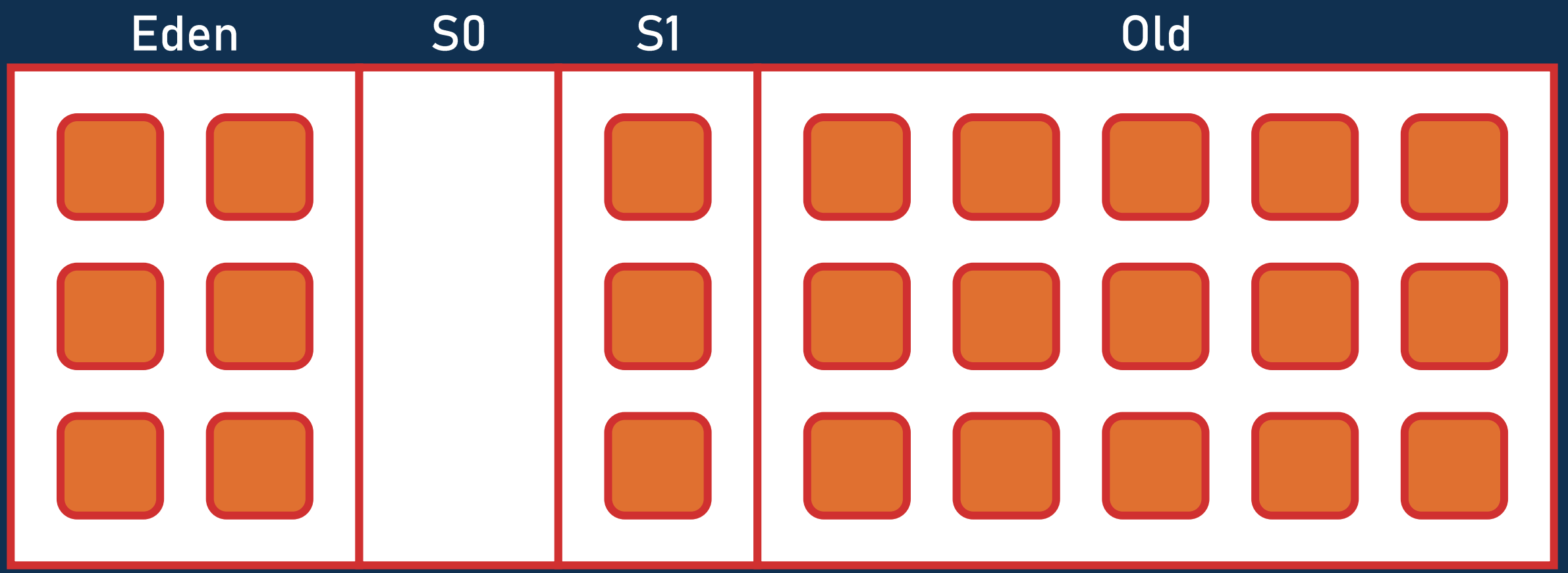
Serial GC

Serial GC

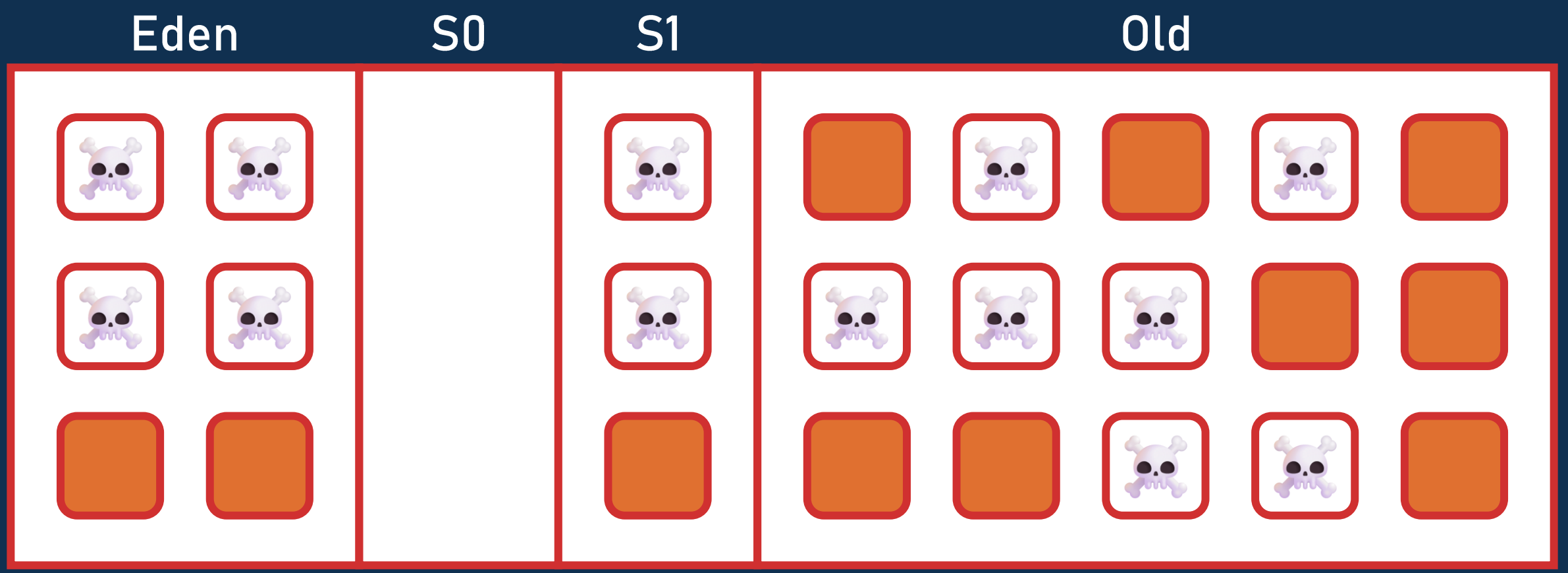
Serial GC



Serial GC – Full GC

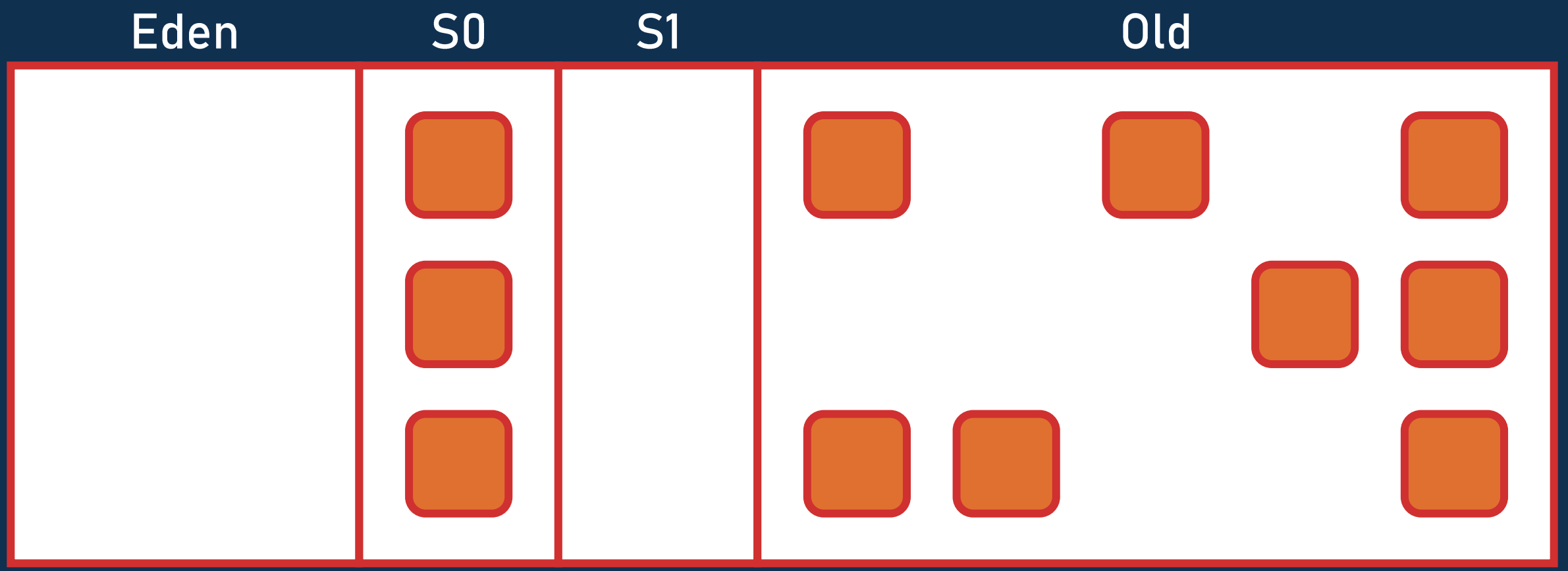


Serial GC – Full GC

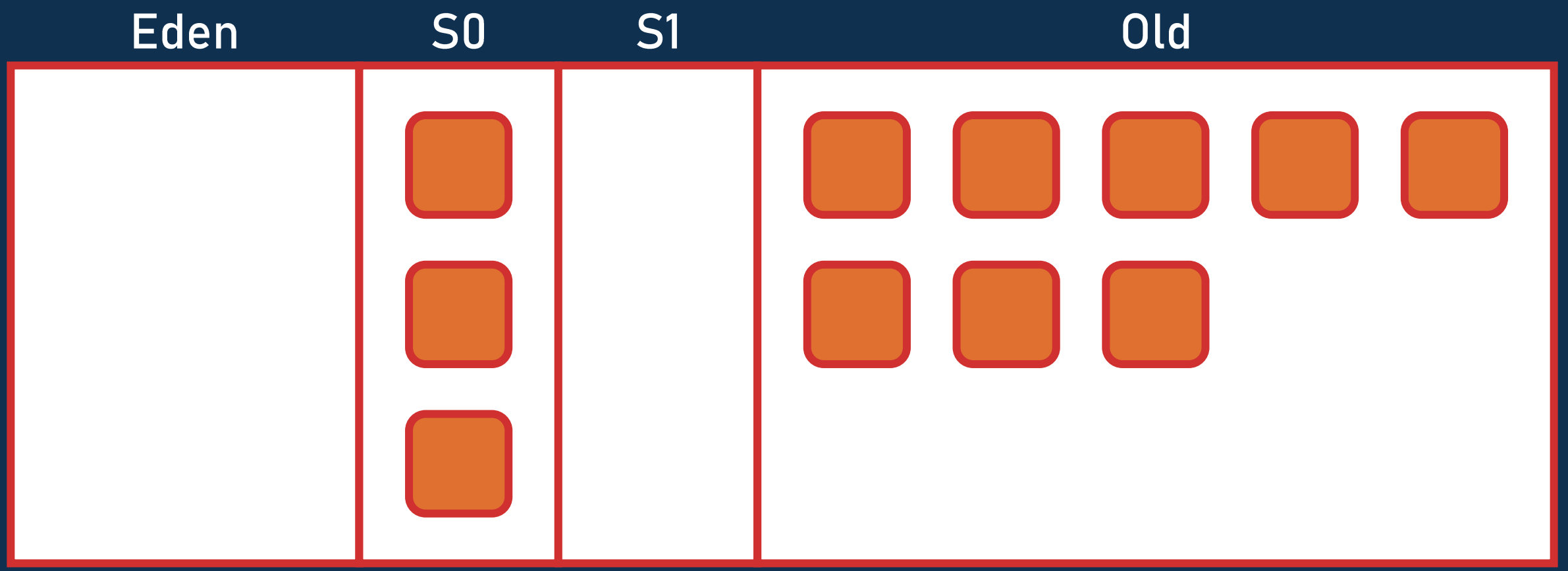


Serial GC – Full GC

Serial GC – Full GC



Serial GC – Full GC



Serial GC

- 💓 Toujours présent et maintenu
- ⛔ Stop-the-world
- ☝️ Un seul thread
- 🎯 Le GC avec le moins d'overhead
- 🤏 Par défaut si < 2 Go de RAM ou < 2 CPUs
- 🐜 Idéal pour les microservices

Parallel GC

Java 5 (2004)

Parallel GC

- 🔀 La même chose mais en parallèle
- ⛔ Toujours stop-the-world
- ⏳ Pauses de plusieurs secondes sur grosses heaps
- 🚀 Le GC avec le plus de throughput
- 📄 Idéal pour le traitement de données, batchs, …

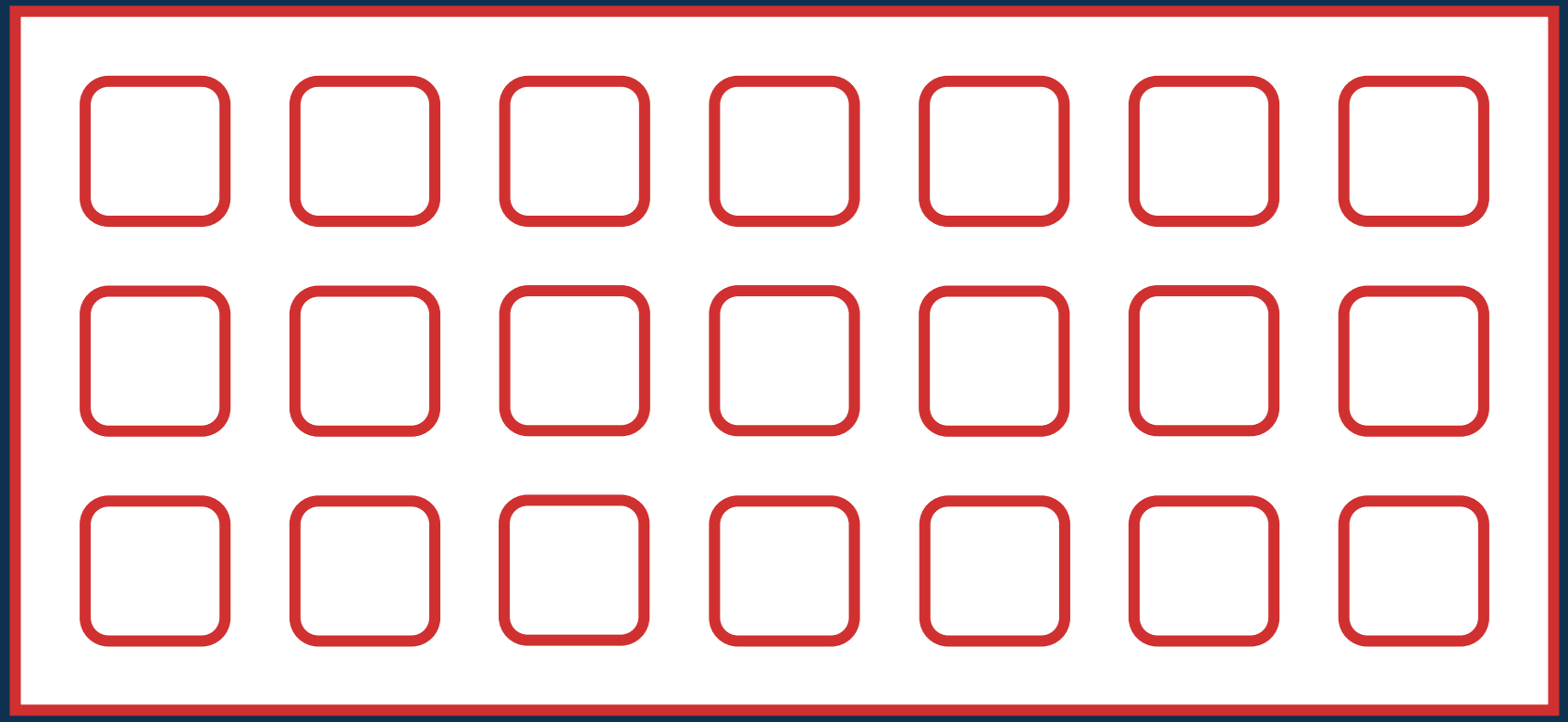
Latence ?

Et la latence ?

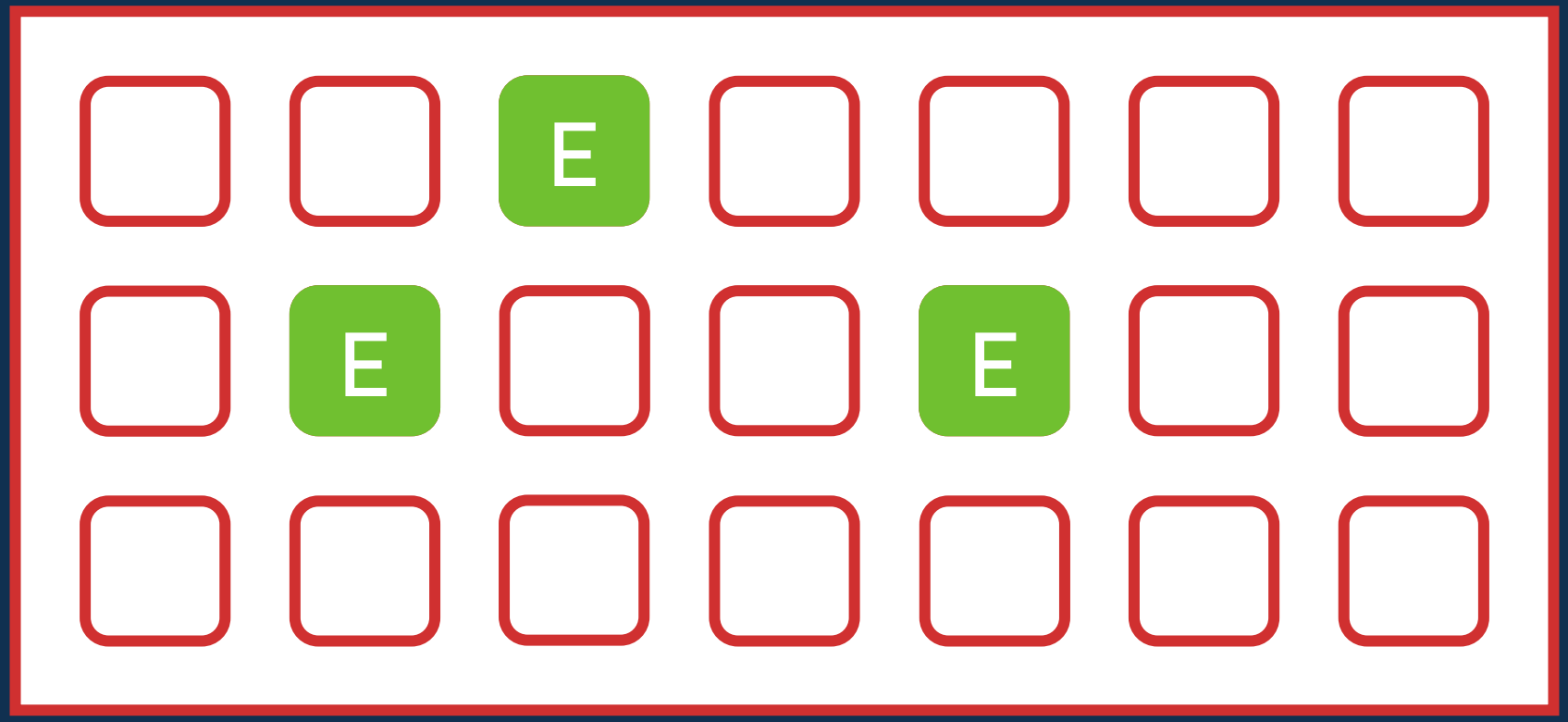
- 🕓 Le GC peut bloquer l'application plusieurs secondes
- 🚂 Essai raté : Train GC
- 🧀 Essai semi-raté : CMS GC
- 🗑️ Essai réussi : G1 GC en Java 7 (défaut en Java 9)

G1 GC
Java 7 (2011)

G1 GC

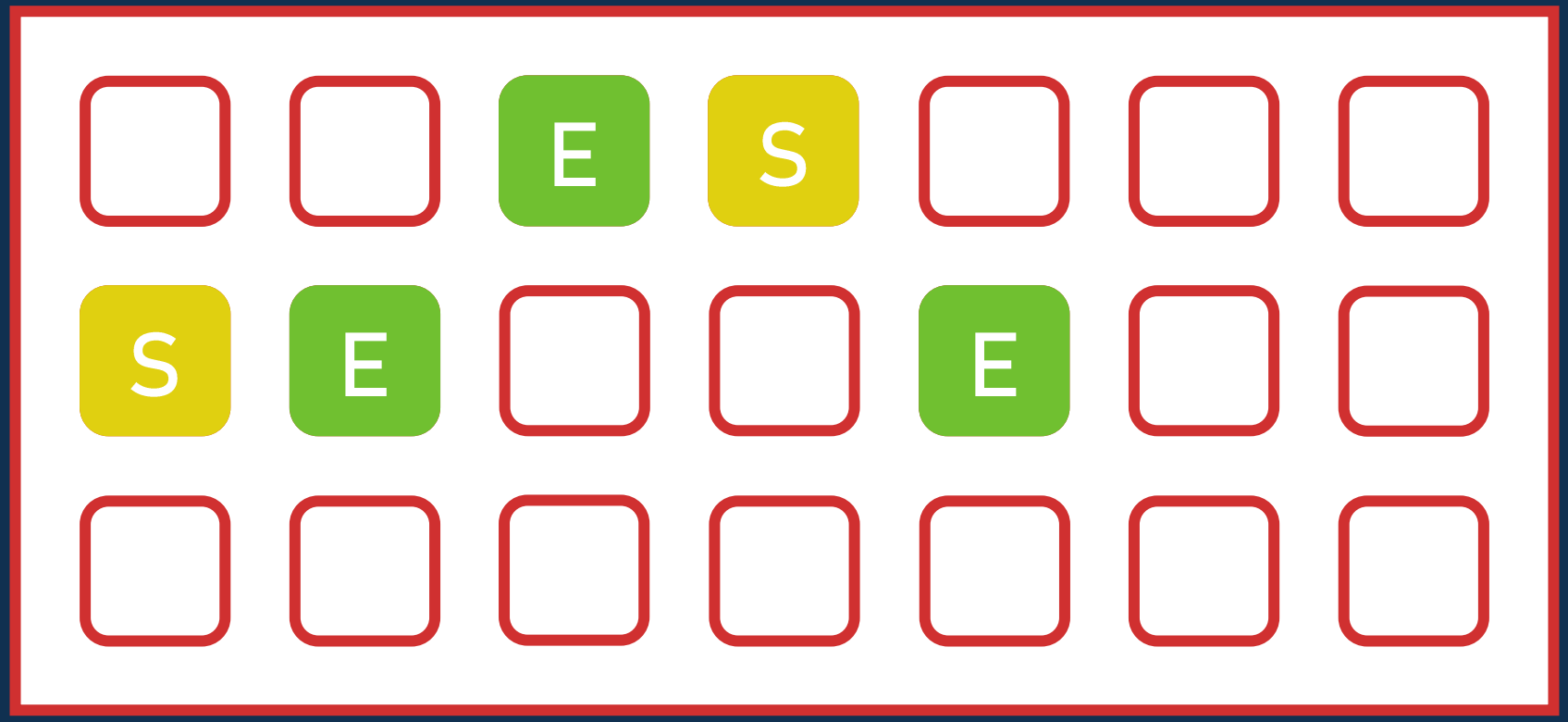


G1 GC



E Eden

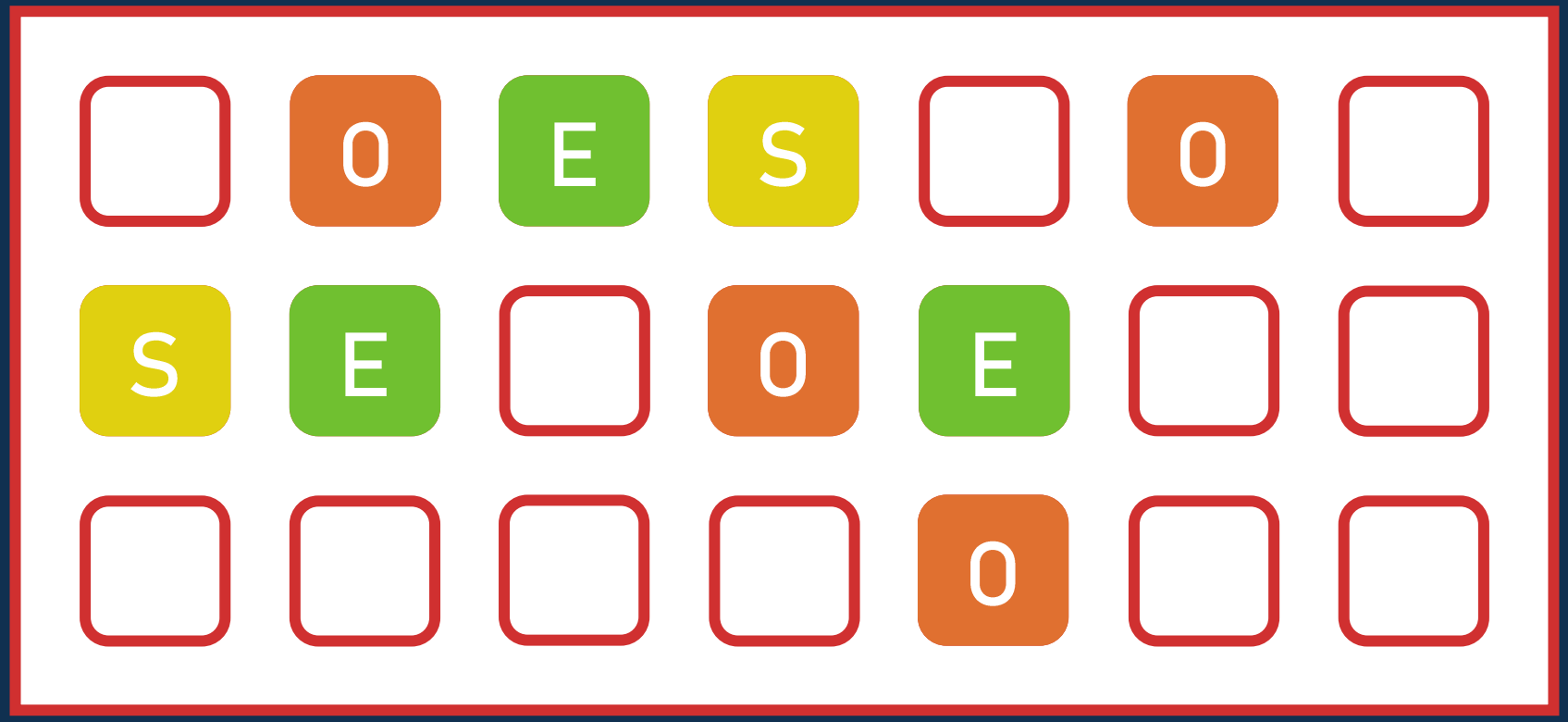
G1 GC



E Eden

S Survivor

G1 GC



E Eden

S Survivor

O Old

G1 GC

E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection

E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection

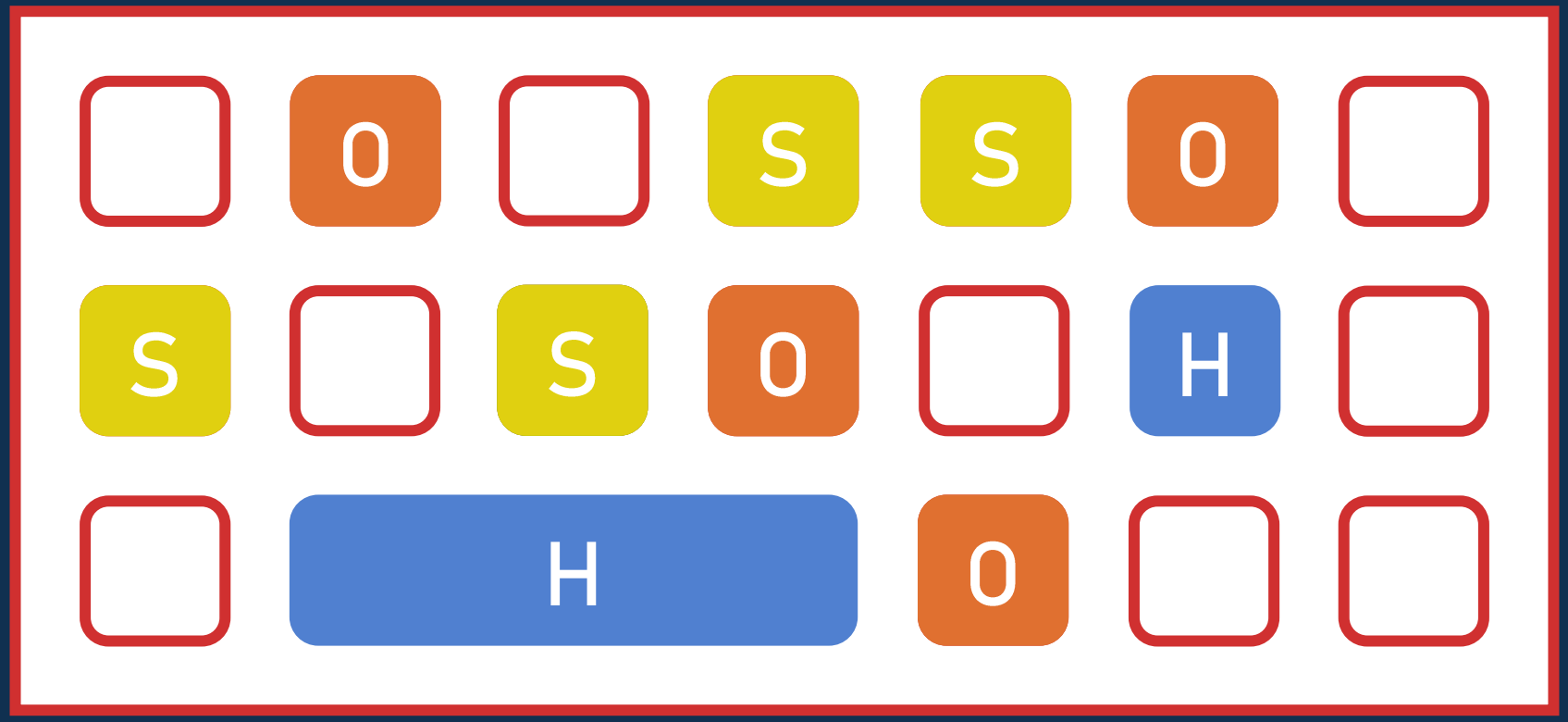
E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection



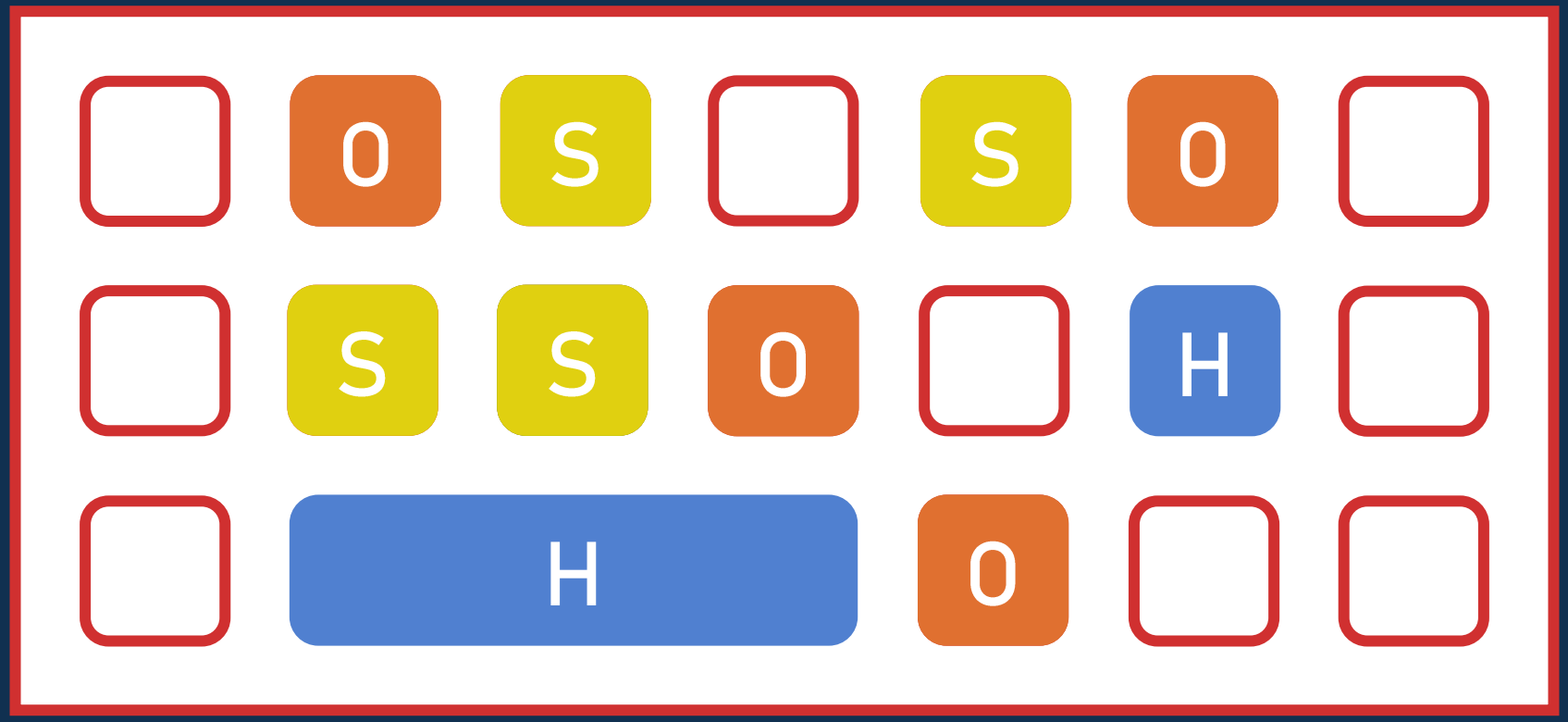
E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection



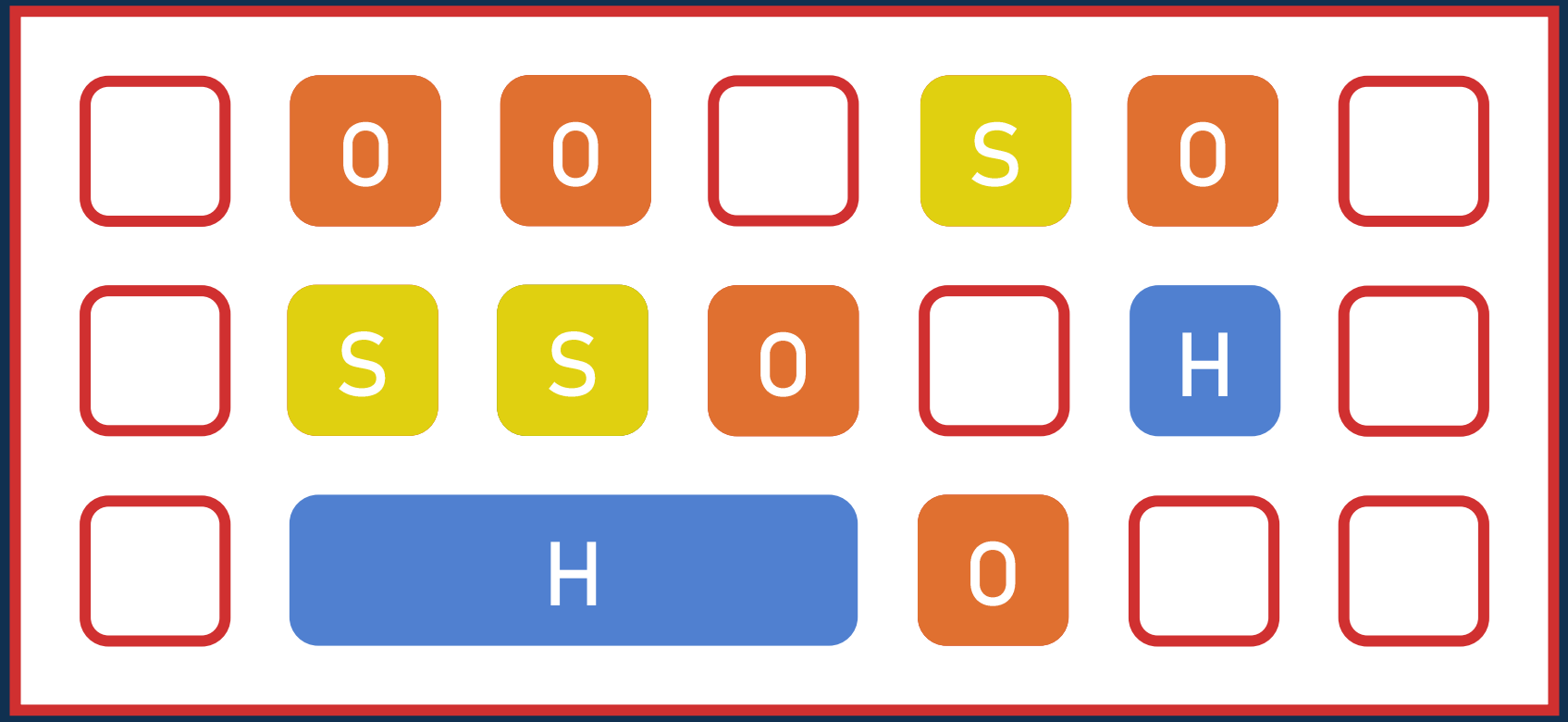
E Eden

S Survivor

O Old

H Humongous

G1 GC – Young collection



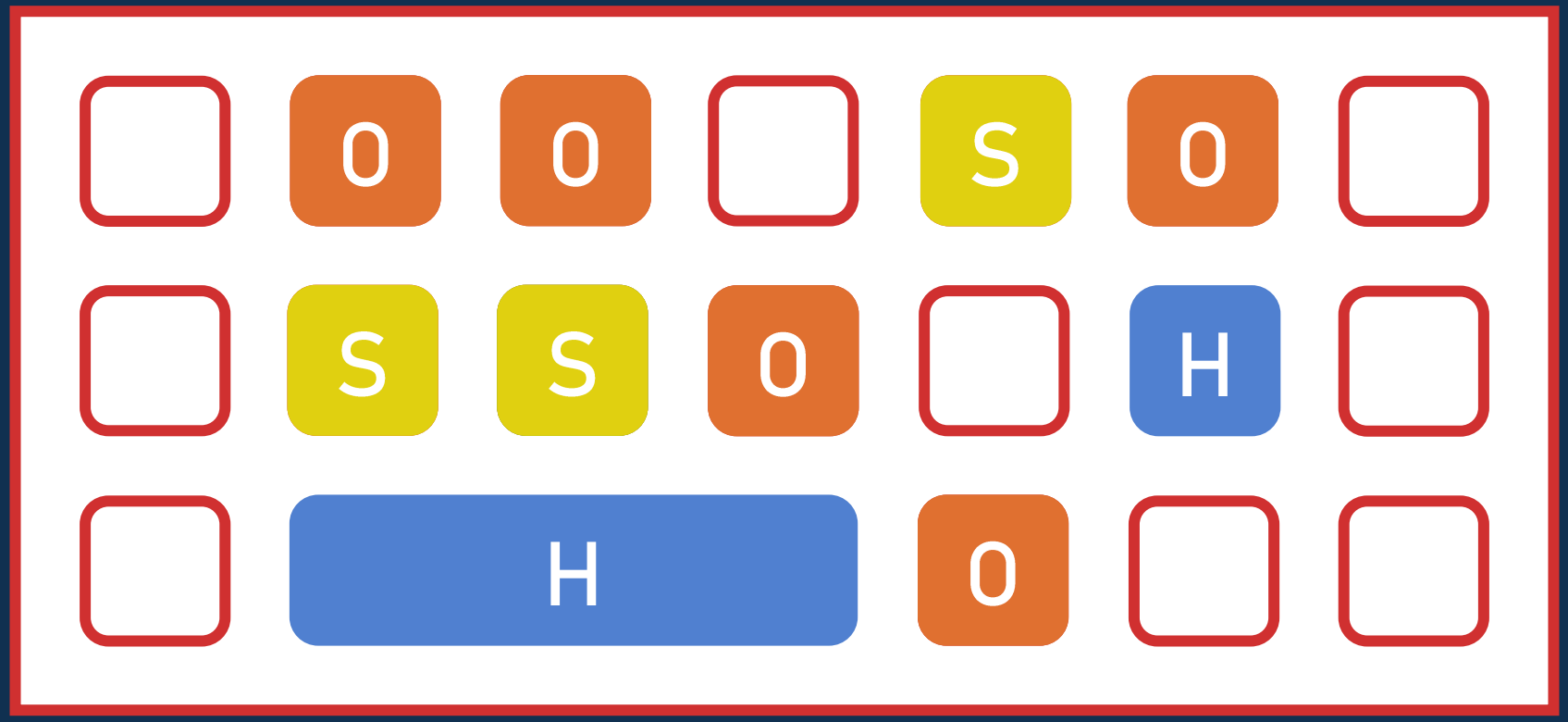
E Eden

S Survivor

O Old

H Humongous

G1 GC – Mixed collection



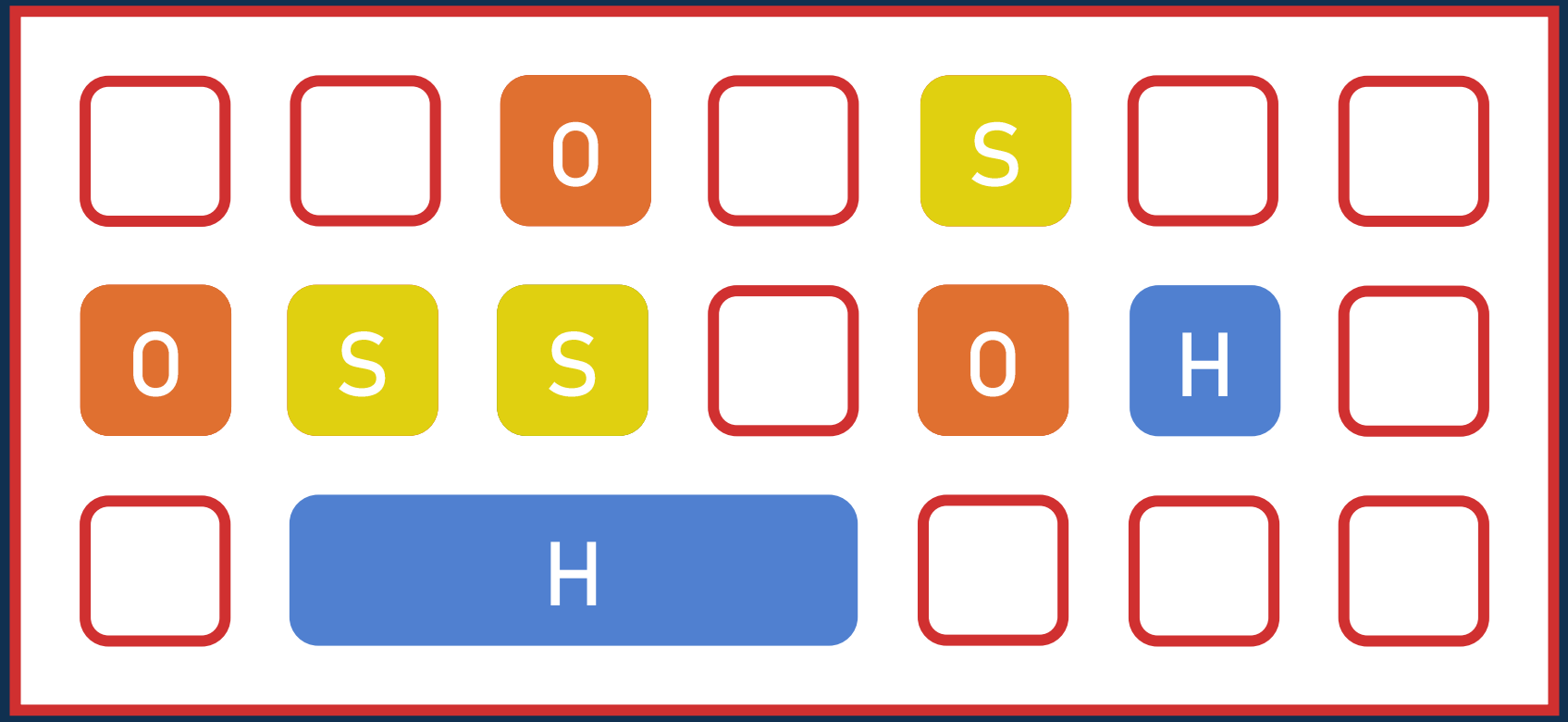
E Eden

S Survivor

O Old

H Humongous

G1 GC – Mixed collection



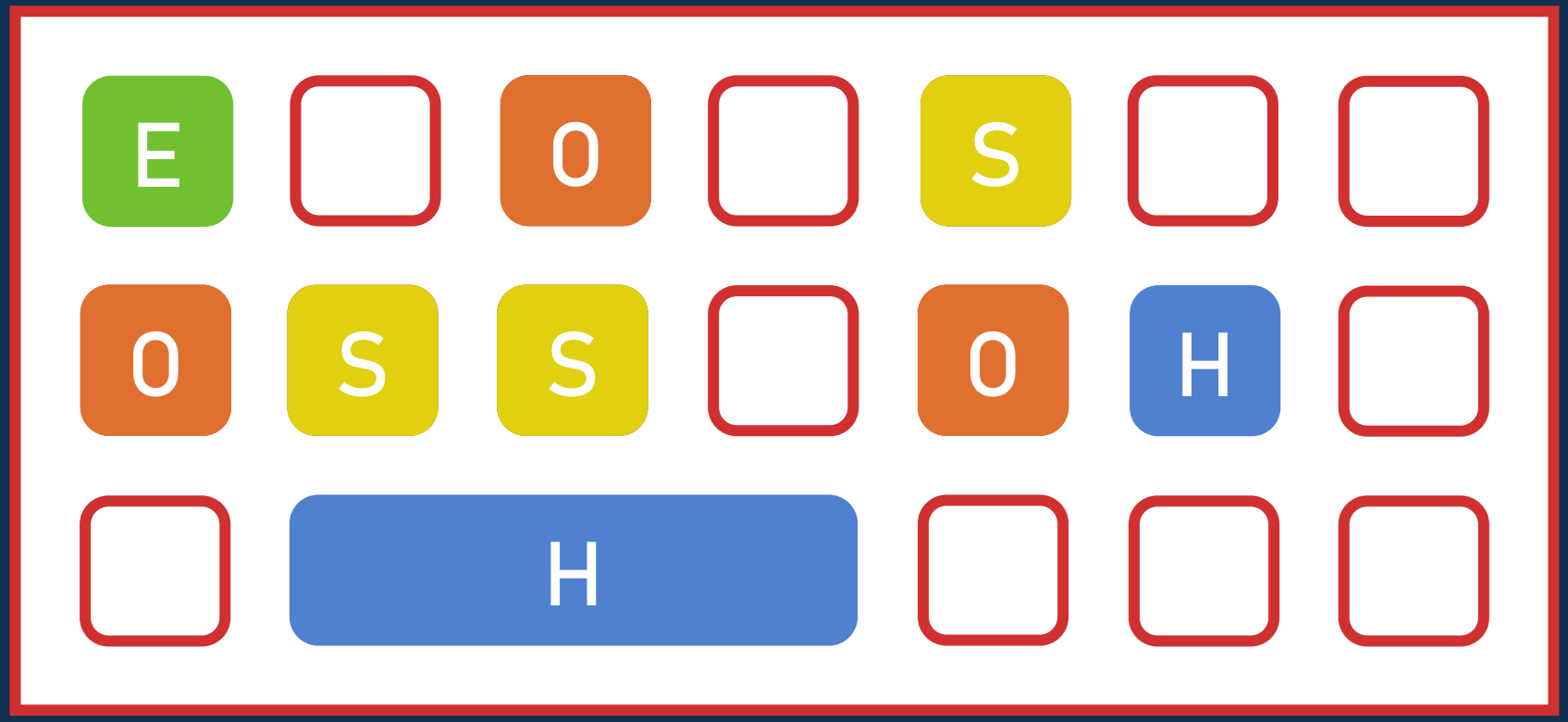
E Eden

S Survivor

O Old

H Humongous

G1 GC



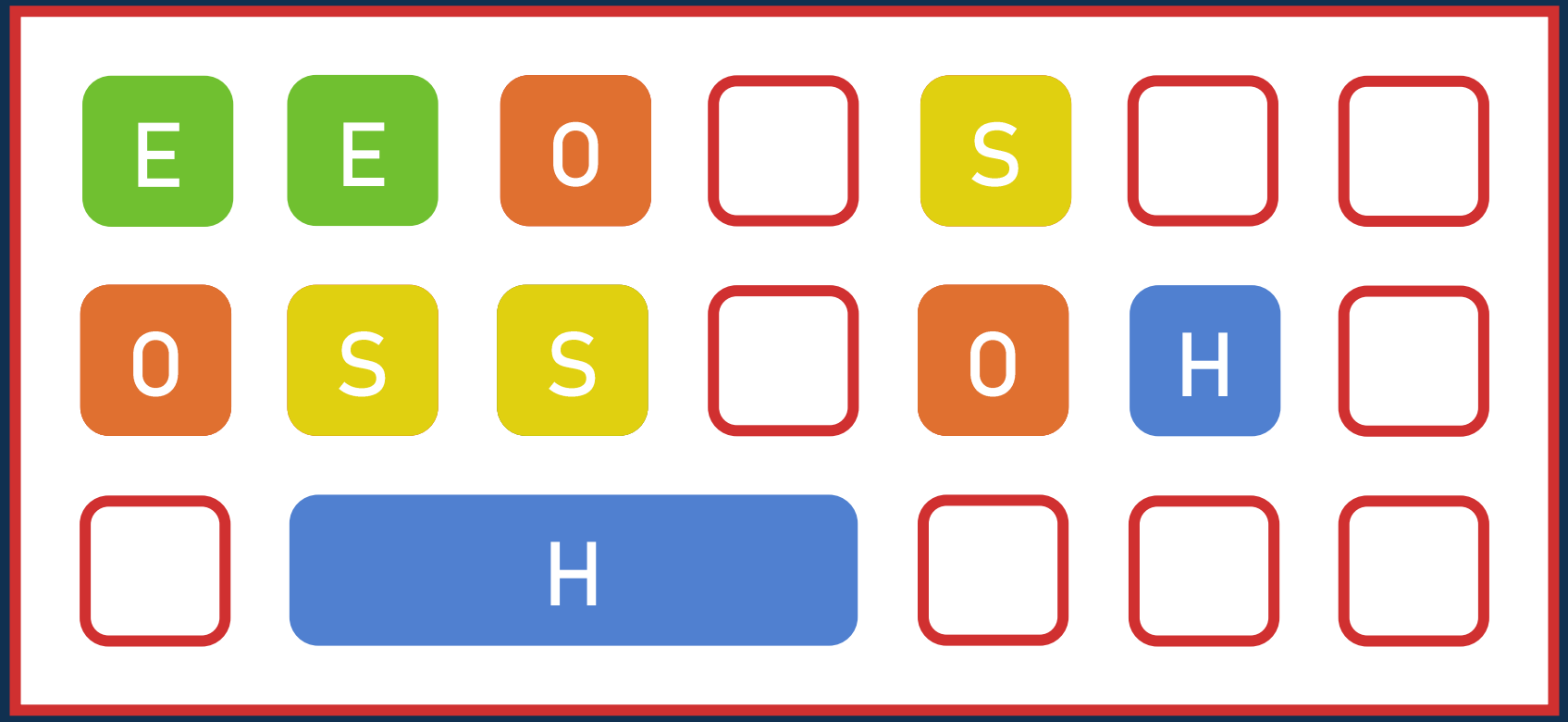
E Eden

S Survivor

O Old

H Humongous

G1 GC



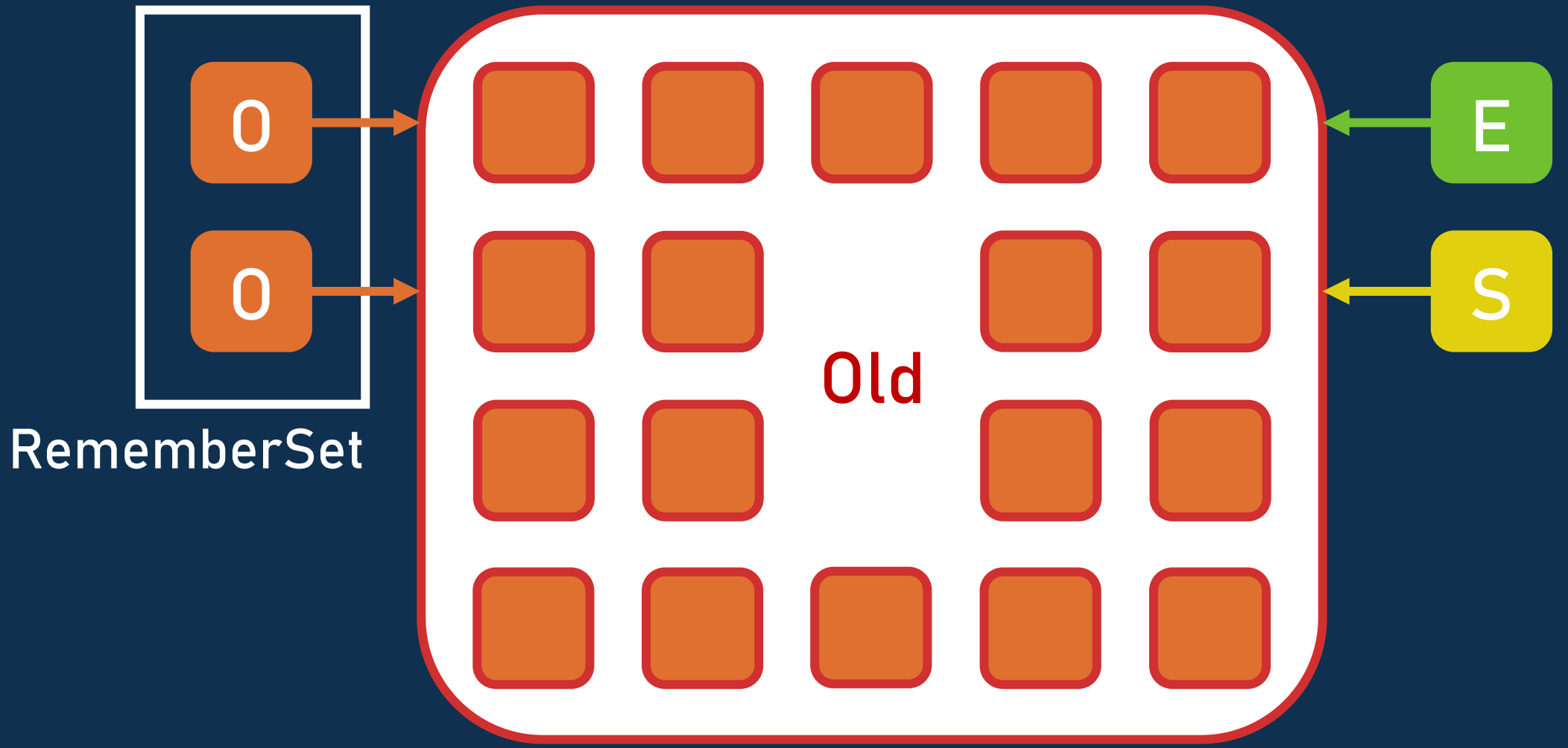
E Eden

S Survivor

O Old

H Humongous

G1 GC – Région



G1 GC – Région

G1 GC

- 🗑️ Il commence par les régions avec le plus de déchets
- 📋 Il copie les objets vivants dans d'autres régions
- ⚠️ Partiellement stop-the-world
- ⏱️ Il s'arrête après 200 ms
- 👌 C'est le GC généraliste

Latence ?

Latence

⏱️ Pause maximum: 200 ms **

⏳ * Il peut faire un full GC

😨 ** C'est par JVM

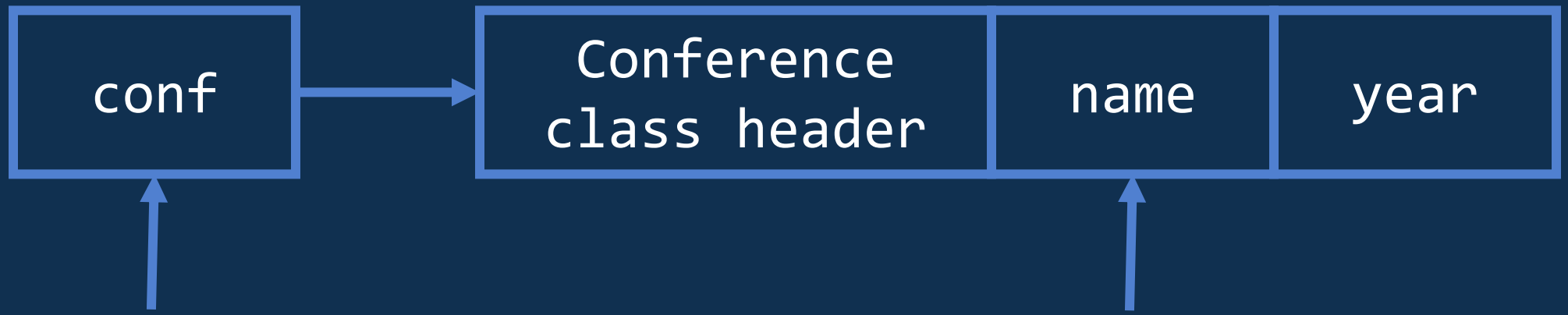
On peut enchainer les GCs sur les micro-services

Ultra low latency
ZGC et Shenandoah GC

ZGC

💡 Idée: stocker des informations dans les références

```
var conf = new Conference("Conférence", 2026);
```



ZGC – Référence

Adresse : jusqu'à 16 To

Flag Finalizer

Flag Remapped : l'adresse est bonne

Flags Mark : il faut récupérer la bonne adresse

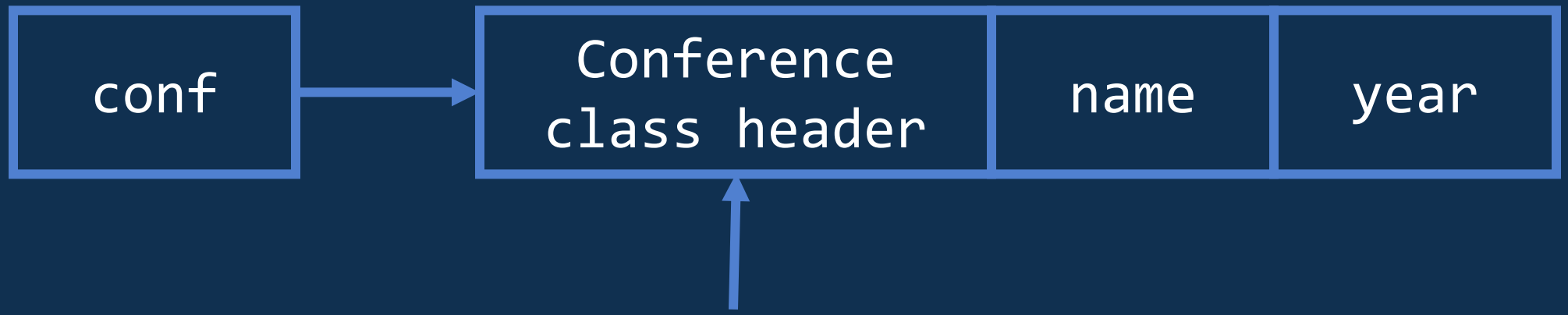
ZGC

- ⚙️ Auto-tune
- 🔄 Le thread applicatif peut faire la mise à jour
- 🐘 Références de 64 octets
- ⏸️ Allocation stalls
- 🧑‍🤝‍🧑 Générationnel depuis Java 21 via flag
  (non-générationnel supprimé en Java 24)

Shenandoah GC

💡 Idée: stocker des informations dans les headers

```
var conf = new Conference("Conférence", 2026);
```



Shenandoah GC – Header

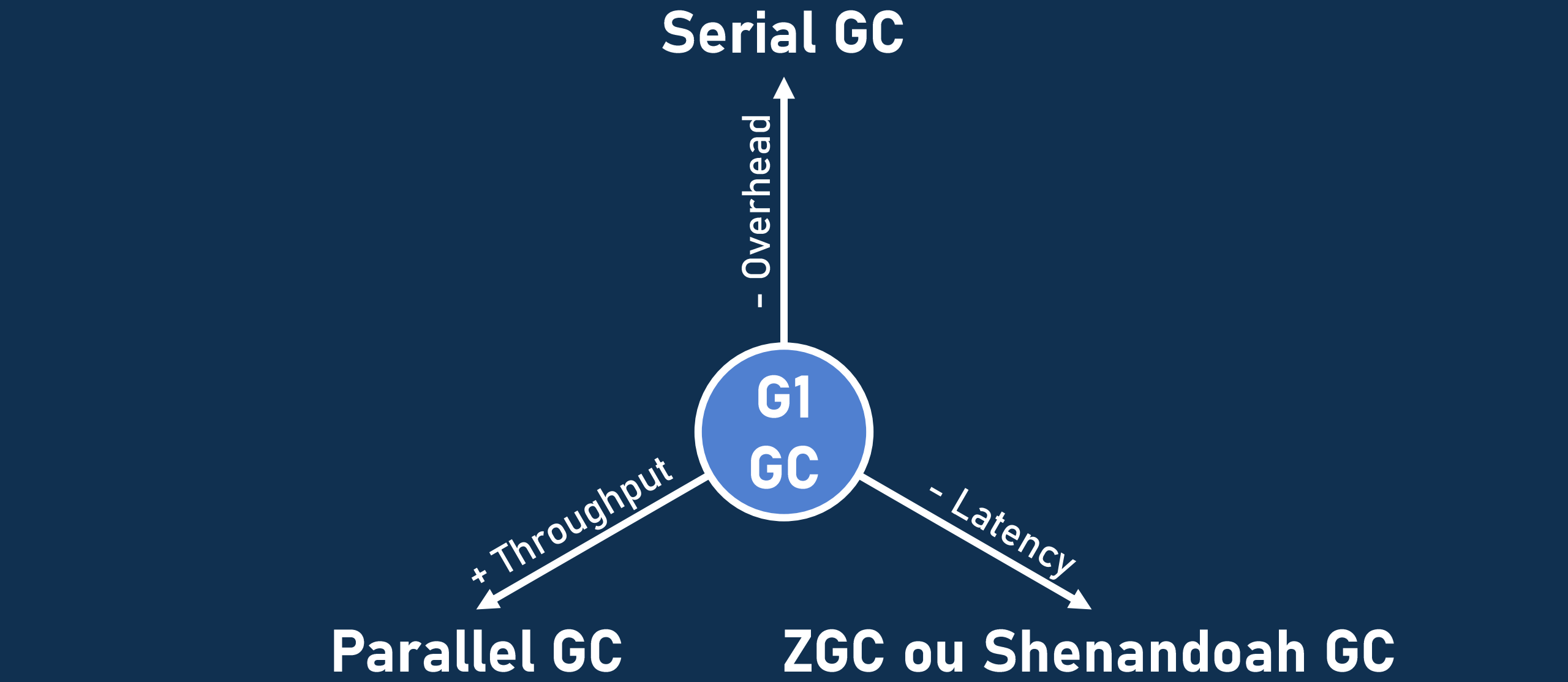
Tags : 01=unlocked, 00/10=locked, 11=GC

Shenandoah GC

- ⚙️ Très configurable
- 📜 Pas de table de correspondance
- ♻️ Références toutes mises à jour avant réutilisation
- ⏸️ Allocation stalls
- ⏸️ Full GC
- 👨‍👨‍👦 Générationnel depuis Java 25

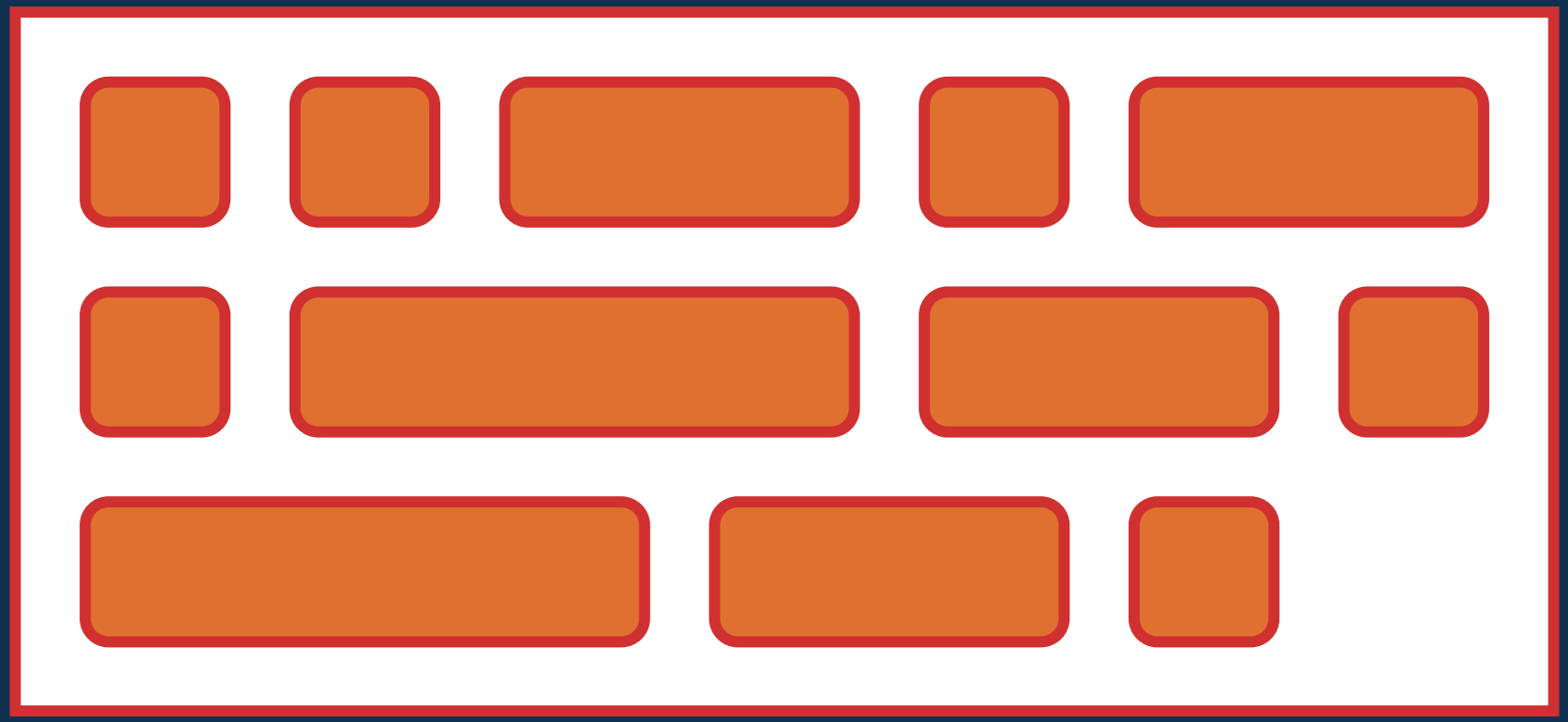
En résumé

Quel GC choisir ?

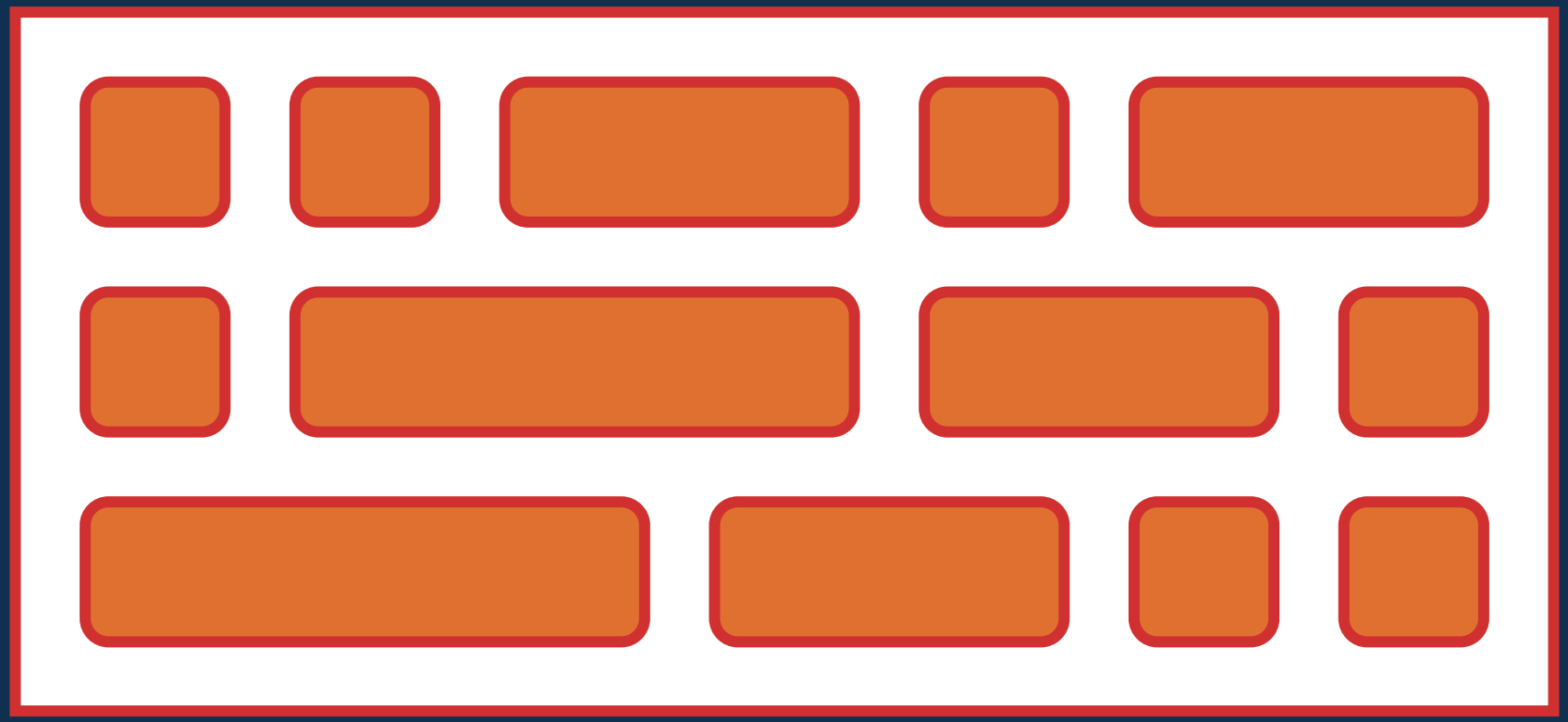


Epsilon GC ?

Epsilon GC



Epsilon GC



En résumé

Quel GC choisir ?

- 🐜 Micro-service (< 2 Go) → Serial GC
- 💻 Serverless, CLI → Epsilon GC
- 📄 Batchs, files, Kafka streams → Parallel GC
- ⏱️ Latence est critique → ZGC ou Shenandoah GC
- 👌 Tous les autres cas → G1 GC

En Java natif

Java natif

Quarkus

Serial GC

Epsilon GC

Graal VM

Serial GC

Epsilon GC

G1 GC
(version enterprise)

Le futur

Le futur

- 💪 Amélioration du throughput de G1 (JEP-522, Java 26)
- 👌 G1 GC remplace Serial GC par défaut (JEP-523)
- 🔮 Bootstrap plus rapide de ZGC
- 🔮 Dimensionnement automatique de la heap (ZGC, G1, Serial GC)

Performances

Pour améliorer les performances

- 👌 Choisir le GC adapté à son besoin
- 👌 Mesurer
- 👌 Avoir une JVM récente

Flags magiques

- 👌 -Xms = -Xmx
- 👌 -XX:+UseLargePages
- 👌 -XX:+AlwaysPreTouch
- 👌 Rien d'autre

Merci pour votre écoute

Antoine DESSAIGNE

Slides

Merci !